

Universidad de las Américas
Facultad de Ingeniería y Ciencias Aplicadas
Ingeniería de Software

Programación IV

*Sistema de monitoreo y análisis de telemetría en videojuegos mediante integración no
invasiva entre Unity, backend y dashboard web*

Alisson Armas
Samuel Cobo
Julio Mera
Amelia Povea
Jeremy Tomaselly

Profesor: PhD (c). Luis Fernando Aguas Bucheli

06 de julio de 2026
Quito, Ecuador

Tabla de contenido

Tema del proyecto.....	4
Objetivo.....	6
Objetivo general.....	6
Objetivos específicos	6
Contextualización	7
Identificación del problema	7
Análisis global, cultural, social, económico y ambiental.....	8
Implementación.....	10
Herramientas utilizadas.....	10
Arquitectura central del sistema web.....	10
Topología de despliegue	12
Modelado relacional y persistencia.....	13
Especificación de la web API	13
Seguridad y blindaje	15
Modelo de amenazas	15
Autenticación servidor-servidor	15
Tokens de sesiones de juego.....	16
Autenticación, roles, y matriz de autorización.....	16
Diccionario exhaustivo de ventanas e interfaces web, controlador por controlador y vista por vista.....	16
Modulo Auth.....	17
Módulo Home.....	23
Módulo Ecosystem	26
Módulo Dashboard	28
Módulo LiveOperations.....	30
Módulo Alerts	31

Módulo Builds	33
Módulo Installations	34
Módulo SessionEvents	36
Módulo SessionEventAnalytics.....	37
Módulo Users	38
Módulo Profile.....	39
Módulo StudioApplications.....	43
Módulo PatchNotesAdmin	46
Módulo Shared	48
Operación, telemetría, alertas y administración.....	53
Integraciones con API's	54
Discord API / Webhooks	54
GitHub API.....	54
Google APIs.....	55
SMTP.....	55
SignalR	55
Atributos de calidad, observabilidad, rendimiento, mantenibilidad y recuperabilidad	55
Repositorio.....	57
Referencias bibliográficas.....	58

Tema del proyecto

Para abarcar la complejidad técnica de este proyecto integrador, hemos estructurado a nuestro equipo de la siguiente manera. Dark Sky Studios es el nombre de nuestro equipo que por un lado desarrolla el videojuego Cosmic Diner y por otro desarrollará este proyecto que será conocido como EchoConsole. EchoConsole se relaciona directamente con Cosmic Diner, pues se encargará de mostrar en tiempo real información como: cuántos jugadores están conectados y cuántas veces se ha instalado el juego. A continuación, presentamos la identidad de nuestras dos divisiones y sus respectivos productos.

EchoConsole es el producto estrella de nuestra división de datos y el puente directo hacia Cosmic Diner. Es el dashboard web construido en ASP.NET Core que utiliza SignalR para mostrar en tiempo real la información de las sesiones de juego. El nombre nace de un concepto muy específico: cada vez que un jugador inicia una partida o interactúa con nuestro juego Cosmic Diner, genera un "eco" de datos que viaja por la red hasta nuestra consola central.

Echo Console es la plataforma web principal de Dark Sky Studios. La solución combina un host ASP.NET Core 8 MVC orientado a interacción humana, identidad, administración y catálogo comercial, con un segundo host ASP.NET Core 8 Web API dedicado a ingesta transaccional, operaciones administrativas servidor-servidor, procesamiento asíncrono y distribución de eventos SignalR. Desde el punto de vista de negocio se percibe como un único centro neurálgico; desde el punto de vista de despliegue es una plataforma de dos procesos web coordinados.

La arquitectura usa Entity Framework Core 8 y SQL Server como sistema de registro para identidad, sesiones, instalaciones, telemetría, builds, alertas y catálogo de herramientas. Un contexto adicional SQLite mantiene notas de parche dentro del proceso API. El catálogo GoldenCat se modela en Tercera Forma Normal mediante StudioApplications y ApplicationScreenshots, con borrado en cascada, visibilidad, clasificación de icono y un índice único filtrado que materializa en SQL Server la regla de exactamente un icono por aplicación.

El sistema de localización comercial ya no depende de llaves de recursos editadas manualmente por un administrador. NameEn/NameEs, PitchEn/PitchEs y DescriptionEn/DescriptionEs se almacenan directamente en SQL Server. StudioApplicationService selecciona el idioma a partir de CultureInfo.CurrentCulture.Name y proyecta strings listos para la vista, manteniendo los archivos RESX únicamente para el marco de interfaz estable.

La API de telemetría implementa registro de instalaciones, ciclo de vida de sesiones, heartbeats, eventos JSON tipados y alertas. Los límites incluyen 32 KiB por solicitud de evento, 4.000 caracteres y 8.192 bytes UTF-8 por payload, profundidad JSON 16, catálogo cerrado de ocho tipos de evento y limitadores de frecuencia. Las operaciones administrativas se protegen con una API key transmitida en X-Admin-API-Key y comparada en tiempo constante. Las sesiones del juego usan tokens aleatorios de 256 bits almacenados únicamente como SHA-256.

Objetivo

Objetivo general

Desarrollar EchoConsole, una plataforma web de monitoreo y análisis para el videojuego Cosmic Diner, que permita centralizar, procesar, almacenar y visualizar en tiempo real la información de telemetría generada por el juego, con el propósito de facilitar el seguimiento de las sesiones, las instalaciones y el comportamiento de los jugadores, proporcionando información confiable que apoye la toma de decisiones durante el desarrollo, mantenimiento y mejora continua del videojuego.

Objetivos específicos

Diseñar e implementar una arquitectura de software basada en ASP.NET Core, SignalR, Entity Framework Core y SQL Server, que permita recibir, validar, almacenar y administrar de manera segura la información de telemetría generada por Cosmic Diner, incluyendo datos relacionados con instalaciones, sesiones de juego, eventos, alertas, versiones del videojuego y demás registros necesarios para el funcionamiento de la plataforma.

Desarrollar un dashboard web interactivo que centralice la información proveniente del videojuego y presente en tiempo real indicadores relevantes, como la cantidad de jugadores conectados, el número de instalaciones, el estado de las sesiones activas, las diferentes builds disponibles y otros datos de interés, mediante una interfaz intuitiva que facilite el monitoreo del sistema y la interpretación de la información por parte de los desarrolladores.

Implementar mecanismos de comunicación en tiempo real, procesamiento de datos, validación de información y controles de seguridad que garanticen la integridad, disponibilidad y confiabilidad de los datos recopilados, permitiendo generar métricas útiles para comprender el comportamiento de los jugadores, detectar situaciones relevantes durante la ejecución del videojuego y respaldar decisiones orientadas a mejorar la experiencia de usuario y el proceso de desarrollo del producto.

Contextualización

Identificación del problema

Los videojuegos forman parte del día a día. Mientras alguien intenta superar niveles en Candy Crush durante un momento libre, otra persona compite en partidas intensas de Call of Duty buscando la victoria. Esta presencia constante demuestra cómo el gaming se ha integrado en la rutina de millones de personas en todo el mundo. Este fenómeno no solo es evidente en la vida cotidiana, sino también en los datos. “El tamaño del mercado mundial de juegos en línea se valoró en 225,28 mil millones de dólares en 2025 y se proyecta que crecerá de 244,68 mil millones de dólares en 2026” (Fortune Business Insights, 2026). Este crecimiento se debe, en gran medida, a la adopción de nuevas tecnologías y dispositivos. Como el auge de los teléfonos inteligentes y el desarrollo de los juegos en la nube, lo que ha ampliado significativamente el acceso a los videojuegos (Fortune Business Insights, 2026). Como consecuencia, se ha generado un aumento considerable en la cantidad de datos producidos por los usuarios.

En este contexto, surge el análisis de juegos, entendido como “los datos y la información que ayudan a mejorar la toma de decisiones en el diseño de juegos, la monetización, la experiencia de juego y el crecimiento empresarial” (Rosenfelder, 2024). Así, los videojuegos modernos no solo funcionan como productos de entretenimiento, sino también como sistemas distribuidos que generan información valiosa para su mejora continua.

A partir de esta realidad, se vuelve evidente que no basta con recolectar grandes volúmenes de datos, sino que es necesario transformarlos en información útil. Los datos no deben limitarse a una representación visual atractiva, sino que deben aportar valor tanto a los desarrolladores como a los usuarios que interactúan con el sistema. Para una empresa que desarrolla videojuegos, comprender el comportamiento de sus jugadores es fundamental, ya que constituye una forma directa de mantenerse en contacto con ellos y evitar que sus necesidades queden en segundo plano. Sin embargo, no siempre resulta eficiente analizar esta información a través de herramientas técnicas como consolas dentro de entornos de desarrollo, ya que estas suelen ser limitadas y poco accesibles.

En cambio, una plataforma más estructurada permite una visualización más completa y comprensible. En este sentido, como equipo en etapa inicial, la implementación de prácticas de análisis de datos resulta clave para fortalecer el proceso creativo, el mantenimiento del videojuego y su posible crecimiento. Su objetivo principal es ofrecer la mejor experiencia

posible al usuario, asegurando que este se sienta escuchado. Como señala Rosenfelder (2024): “la creatividad humana combinada con datos excepcionales produce juegos increíbles”. Además, el mismo autor sugiere que una de las mejores formas de iniciar con el análisis de datos es mediante sistemas que integren tanto información histórica como actual del videojuego; lo que refuerza la necesidad de contar con herramientas que gestionen estos datos de manera eficiente.

De esta manera, se identifica un problema específico relacionado con la falta de visibilidad sobre aspectos clave del sistema, como el número de jugadores activos y la cantidad de instalaciones del videojuego. Asimismo, existe una dificultad para monitorear múltiples versiones o builds del juego de manera simultánea, lo que limita la capacidad de análisis. Este problema se enmarca en los desafíos propios de los sistemas distribuidos, donde la información se genera desde múltiples fuentes y requiere mecanismos adecuados para su integración y monitoreo.

Análisis global, cultural, social, económico y ambiental

El análisis de este problema también debe considerar diversos factores contextuales. En el ámbito global, se proyecta que el mercado continúe en expansión, alcanzando “501,91 mil millones de dólares en 2034, exhibiendo una tasa compuesta anual del 9,4%” (Fortune Business Insights, 2026). Este crecimiento implica una distribución masiva de software y una mayor necesidad de monitoreo remoto en diferentes plataformas. Mientras que, desde el punto de vista social, se reconoce que “para maximizar los efectos positivos de los videojuegos, es importante centrarse en cómo los diferentes tipos de jugadores desarrollan sus hábitos de juego” (Kaufmann, 2022). Esto resalta la importancia de comprender el comportamiento de los usuarios y adaptar las experiencias digitales en función de sus necesidades.

En el ámbito económico, la toma de decisiones basada en datos se ha vuelto esencial. Como indica Pereira (2023), “en esta era digital, la información precisa y confiable es esencial para guiar las decisiones estratégicas [...] importancia de estos elementos es crucial para que las industrias prosperen en el entorno altamente competitivo del e-commerce”. Esto demuestra que el análisis de métricas no solo permite evaluar el éxito de un producto, sino también optimizar los recursos durante su desarrollo. Por otro lado, en el factor ambiental, aunque de forma más indirecta: “Microsoft estima que una consola de videojuegos moderna promedio emite alrededor de 72 kilogramos de dióxido de carbono al año” (Safdie, 2025). Esto evidencia

la necesidad de gestionar eficientemente los recursos digitales y evitar la sobrecarga de sistemas mediante un monitoreo adecuado.

En cuanto al factor cultural, los videojuegos se han consolidado como un fenómeno diverso. Para algunas personas, representan una forma de relajación tras una jornada extensa; para otras, constituyen un medio de socialización o incluso la demostración de habilidades (Kaufmann, 2022). Esta diversidad en los patrones de uso refuerza la importancia de analizar el comportamiento de los usuarios.

Frente a este panorama, la implementación de una página web como herramienta de monitoreo se presenta como una solución adecuada. Esta permite centralizar la información proveniente del videojuego. Además, su integración como una capa independiente dentro del proyecto garantiza que no interfiera con el gameplay ni con los sistemas existentes. De acuerdo con Rosenfelder (2024), una plataforma como esta:

Ayuda a mejorar todos los aspectos del juego, desde la publicidad hasta la jugabilidad. Cuando se usa correctamente, el análisis de datos ayuda a que los juegos sean más divertidos, lo que reduce la pérdida de clientes y hace que el juego sea rentable. Esto respalda la utilidad de este tipo de soluciones en el contexto del desarrollo de videojuegos.

En síntesis, la propuesta responde a un problema real relacionado con la falta de herramientas accesibles para el monitoreo y análisis de datos en videojuegos. Cuenta con sustento teórico que respalda la importancia del análisis de datos en la industria digital y considera diversos factores contextuales que influyen en su relevancia. Además, aporta valor al permitir una mejor comprensión del comportamiento de los usuarios sin afectar el funcionamiento del sistema, lo que la convierte en una solución pertinente y viable.

Implementación

Herramientas utilizadas

Área	Tecnología	Versión	Uso
Runtime	.NET	Net8.0	Ambos hosts web
Web	ASP .NET Core MVC	8.0	Controladores, Razor, localización y filtros.
API	ASP .NET Core WEB API	8.0	Controladores JSON, ProblemDetails, Swagger
ORM	Entity Framework Core SQL Server	8.0.10	Modelo principal y catálogo.
ORM local	Entity Framework Core SQLite	8.0.10	PatchNotes en EchoConsole.Api.
Identidad	ASP.NET Core Identity	8.0.10	Usuarios, credenciales, tokens, cookies y claims
Tiempo real	SignalR	8.0.10	Telemetría API-Web y actualizaciones a navegadores.
UI	Razor + Tailwind CSS	Tailwind 3.4.x en package-lock	Estética CRT/cyberpunk responsiva.
OAuth	Google Authentication	8.0.10	Inicio de sesión externo opcional
Documentación API	Swashbuckle	10.1.7	Swagger en desarrollo.

Arquitectura central del sistema web

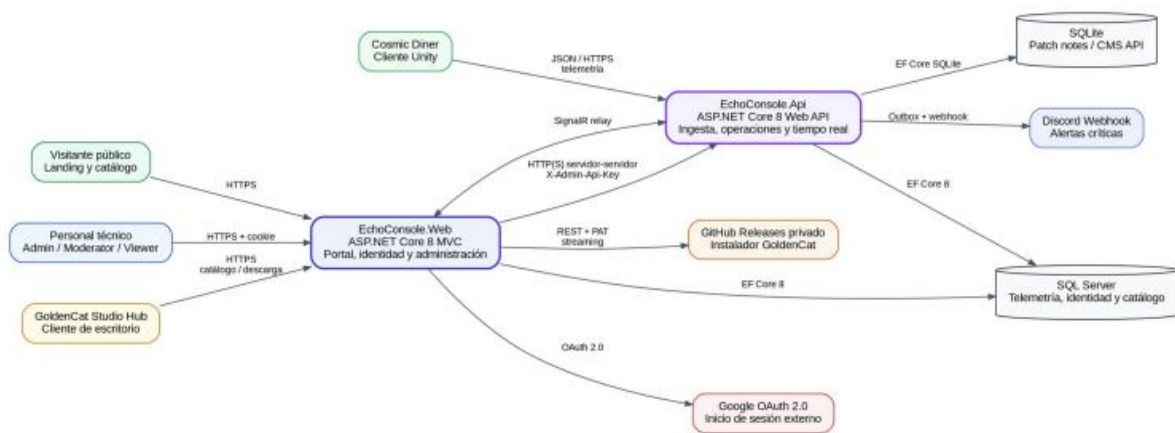
Echo Console implementa un centro de control web para Dark Sky Studios. Su responsabilidad primaria es representar, asegurar y operar el estado del ecosistema: identidad del personal, inventario de instalaciones, builds, sesiones, eventos de juego, alertas, perfiles,

operaciones en vivo, simulación y catálogo de herramientas. La aplicación no es una página de marketing con una API auxiliar; la API y la consola constituyen el producto principal.

La denominación monolito debe precisarse. A nivel lógico la solución comparte dominio, contratos, base relacional y ciclo de administración. A nivel físico existen dos hosts ASP.NET Core 8: EchoConsole.Web y EchoConsole.Api. El primero ejecuta MVC, Razor, Identity y el CMS GoldenCat; el segundo expone Web API, SignalR y workers. Esta separación reduce la superficie pública de los endpoints administrativos y permite escalar la ingesta independientemente de la renderización.

Figura 1

Contexto operativo y fronteras de confianza de Echo Console



Composición de la solución

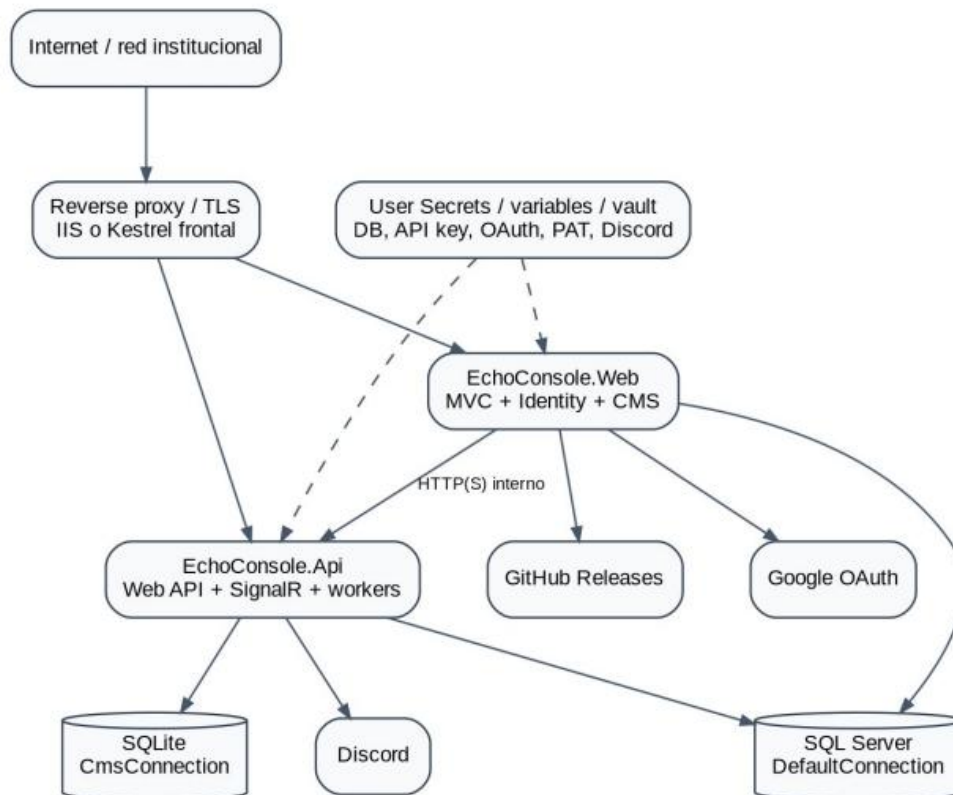
Host	Responsabilidad	Persistencia	Interfaces
EchoConsole.Web	Portal MVC, Identity, sesiones humanas, Razor, catálogo GoldenCat, proxy de descarga y relay SignalR.	SQL Server mediante EchoConsoleDbContext y StudioCatalogDbContext.	HTML/Razor, cookies, hubs /hubs/admin-telemetry y /hubs/profile-telemetry.
EchoConsole.Api	Ingesta de Unity, administración	SQL Server mediante EchoConsoleDbContext; SQLite	REST JSON, Swagger en

	servidor-servidor, consultas, simulación, telemetría, Patch Notes y workers.	mediante ApplicationDbContext.	desarrollo y hub /hubs/telemetry.
SQL Server	Sistema de registro central.	Identidad, telemetría, alertas, catálogo, sesiones y outbox.	TDS mediante EF Core 8.
SQLite	CMS ligero del host API.	PatchNotes.	Archivo local configurado por CmsConnection.

Topología de despliegue

Figura 2

Topología de despliegue recomendada



EchoConsole.Web referencia el proyecto EchoConsole.Api para reutilizar User, EchoConsoleDbContext y contratos, pero sus operaciones administrativas se ejecutan mediante

clientes HTTP. Esta combinación produce acoplamiento de compilación y separación de ejecución. El patrón facilita compartir contratos, aunque convendría extraerlos a un ensamblado Contracts independiente para impedir que el portal dependa de detalles del host API.

Modelado relacional y persistencia

La solución usa tres contextos EF Core con responsabilidades diferentes. EchoConsoleDbContext es el modelo principal y se reutiliza en Web y API. StudioCatalogDbContext vive en el portal y comparte la conexión SQL Server, pero mantiene su propia tabla de historial de migraciones. ApplicationDbContext vive en la API y usa SQLite para las notas de parche.

Contexto	Procedor	Tablas principales	Migraciones
EchoConsoleContext	SQL Server	Users, UserSessions, Installations, GameSessions, GameSessionEvents, GameBuilds, AlertTypeDefinitions, SystemAlerts, AlertDiscordOutboxMessages	Historial EF principal del proyecto API.
StudioCatalogDbContext	SQL Server	StudioApplications, ApplicationScreenshots	StudioCatalogMigrationsHistory
ApplicationDbContext	SQLite	PatchNotes	Historial del CMS API.

Especificación de la web API

EchoConsole.Api es el núcleo transaccional. Recibe clientes no interactivos, valida contratos, persiste estados y emite eventos. También ofrece endpoints administrativos

consumidos por EchoConsole.Web mediante un HttpClient que agrega X-Admin-API-Key. El navegador no recibe esa clave ni llama directamente a los endpoints de administración.

- ProblemDetails y respuestas JSON estructuradas.
- Swagger y Swagger UI solo en desarrollo.
- DbContextPool para SQL Server.
- CORS con orígenes explícitos y credenciales.
- Response compression en producción.
- Rate limiting antes de CORS/autenticación.
- SignalR con filtro de API key.
- Workers de presencia y Discord.

Con la configuración predeterminada, el limitador general admite hasta 300 solicitudes por minuto. El canal de eventos admite 120 eventos por minuto por SessionId mediante una ventana deslizante de 60 segundos y seis segmentos de 10 segundos. Si una sesión emite de forma uniforme, la media autorizada es 2 eventos por segundo; los bursts quedan acotados por la ventana y no se encolan porque QueueLimit es cero.

- Heartbeat recomendado: cada 15 segundos; timeout comunicado al cliente: 45 segundos.
- Cuerpo HTTP máximo para eventos: 32.768 bytes.
- PayloadJson: 4.000 caracteres, 8.192 bytes UTF-8 y profundidad JSON máxima 16.
- La respuesta 429 incluye Retry-After cuando el lease del limitador expone esa metadata.
- La capacidad efectiva de SQL Server y SignalR debe medirse con pruebas de carga; los límites de aplicación no constituyen por sí mismos un SLO.

Seguridad y blindaje

Modelo de amenazas

<i>Activo</i>	<i>Amenaza</i>	<i>Control implementado</i>	<i>Riesgo residual</i>
API administrativa	Uso no autorizado	API key, policy, comparación constante.	Secreto estático; requiere rotación y vault.
Sesión de juego	Suplantación	Token aleatorio de 256 bits, hash SHA-256, comparación constante.	Inicio de sesión no autentica la instalación con un secreto propio.
Telemetría JSON	DoS / payload malicioso	Rate limit, tamaño, profundidad, tipos cerrados, EF parametrizado.	Alert endpoint público puede ser abusado dentro de límites.
Cuenta humana	Robo de cookie	HttpOnly, Secure, SameSite Lax, security stamp y sesión DB.	Requiere CSP y protección XSS continua.
CMS de archivos	Traversal / contenido riesgoso	Ruta generada, extensiones, tamaño, staging, GUID.	SVG externo debe revisarse antes de publicar.
PAT de GitHub	Exfiltración	Solo backend, options, user-secrets, no redirect de Authorization.	Debe rotarse y limitarse al repositorio.
Webhook Discord	Menciones/inyección	HTTPS, allowed_mentions vacío, truncado.	Contenido técnico sigue saliendo a un tercero.

Autenticación servidor-servidor

Los endpoints `/api/admin`, `/api/profile` y el hub `/hubs/telemetry` utilizan la misma clave configurada en `AdminApiSecurity:ApiKey`. `AdminApiKeyAuthenticationHandler` lee `X-Admin-API-Key`, rechaza configuración vacía y compara los bytes en tiempo constante

mediante `CryptographicOperations.FixedTimeEquals`. El principal resultante recibe `Role=ServerAdmin` y `api_access=admin`; la policy exige autenticación y el claim.

Tokens de sesiones de juego

`SessionTokenService` genera 32 bytes criptográficamente aleatorios, los codifica como Base64 URL-safe y elimina padding. El token en claro se devuelve al cliente al iniciar la sesión. SQL Server conserva SHA-256 en `SessionTokenHash`. Heartbeat, eventos y cierre recalculan el hash y comparan en tiempo constante. El resultado separa el identificador público `SessionId` del secreto de autorización.

Autenticación, roles, y matriz de autorización

`EchoConsole.Web` usa `IdentityCore<User>` con clave int y almacén EF Core. Se exige email único, cuenta y email confirmados. La contraseña mínima es de ocho caracteres; dígitos, mayúsculas, minúsculas y símbolos no son obligatorios. Los tokens de Identity viven dos horas y el security stamp se revalida cada minuto.

- Login local: valida usuario, estado, contraseña, confirmación y crea `UserSession` persistida.
- Google: challenge/callback opcional si `ClientId` y `ClientSecret` están configurados.
- Registro: crea User, email de OTP y confirmación antes del acceso.
- Forgot/Reset: token de Identity con vida de dos horas; restablecer revoca sesiones.
- VerifyOtp/ResendOtp: límite de cinco intentos por minuto por IP.
- Logout: revoca la sesión rastreada y elimina la cookie.

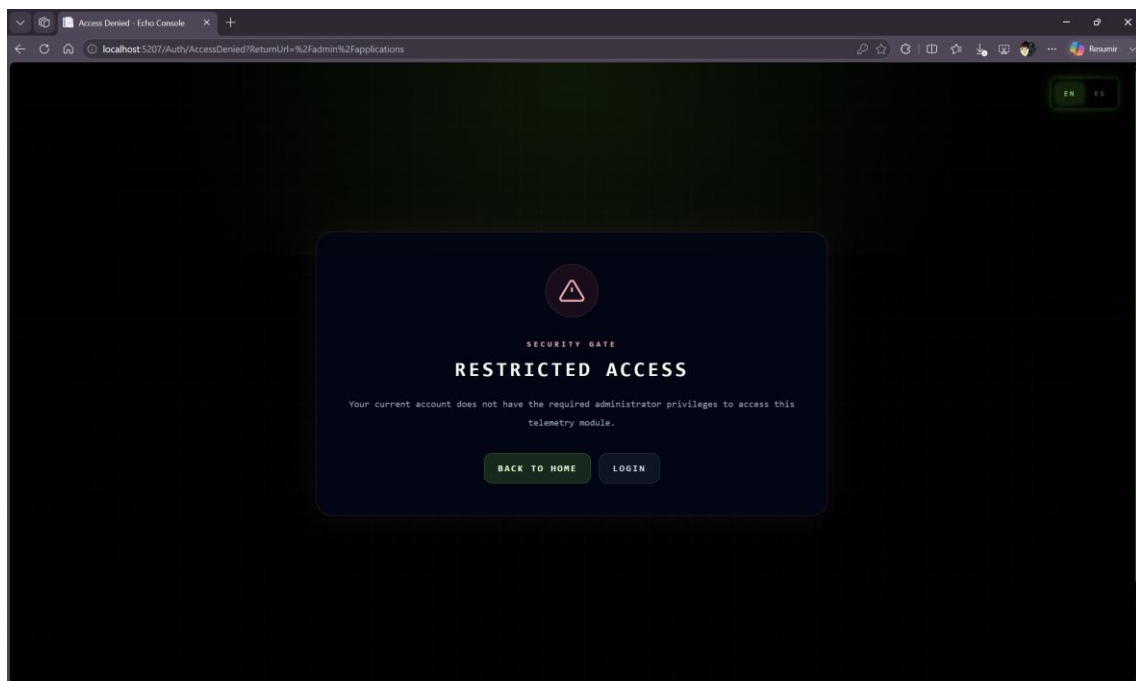
Diccionario exhaustivo de ventanas e interfaces web, controlador por controlador y vista por vista

Las vistas usan un shell Razor compartido con Tailwind CSS. El lenguaje visual combina fondo negro, tipografía monoespaciada, verdes de fósforo, ámbar GoldenCat, rejillas, scanlines y estados luminosos. Los temas del usuario se exponen como claims y clases dinámicas. La accesibilidad se apoya en labels, alt text, foco visible, validación y localización en/en-es.

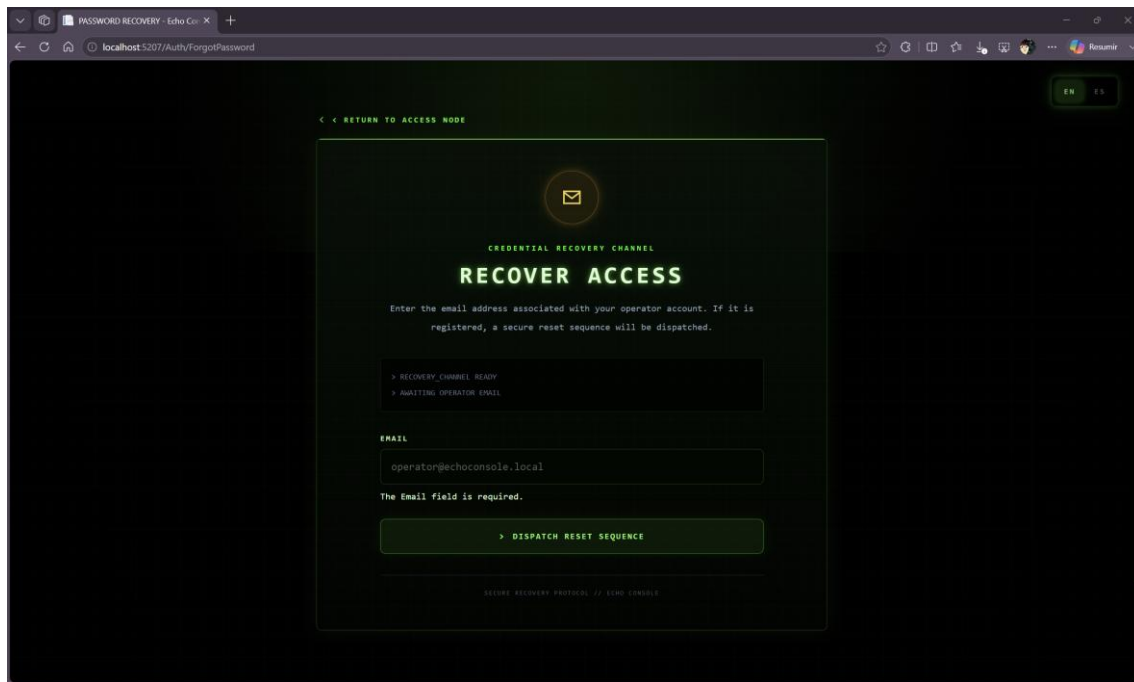
- El `\_\_Topbar` muestra estado, login/perfil y pestañas públicas de ecosistema; oculta tabs en rutas administrativas.
- El `\_\_Sidebar` presenta únicamente módulos aplicables al contexto autenticado.
- Las escrituras usan formularios POST/DELETE con antiforgery.
- La interacción en vivo se implementa con JavaScript modular y SignalR.
- Los formularios complejos usan mensajes DataAnnotations localizados.
- El CSS personalizado de radar/galería se compila en `tailwind.input/output` para no contaminar Razor con keyframes.

Modulo Auth

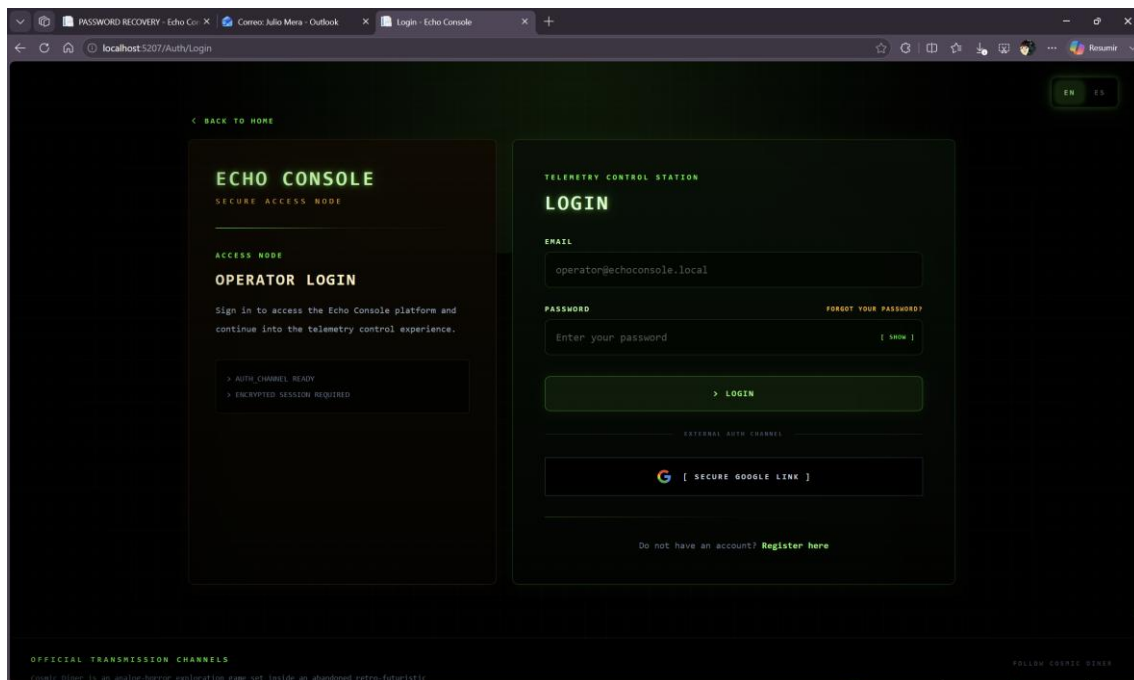
Auth/AccessDenied: Superficie de identidad con layout dedicado, validación localizada y estética de terminal. Evita exponer el shell administrativo antes de autenticar.

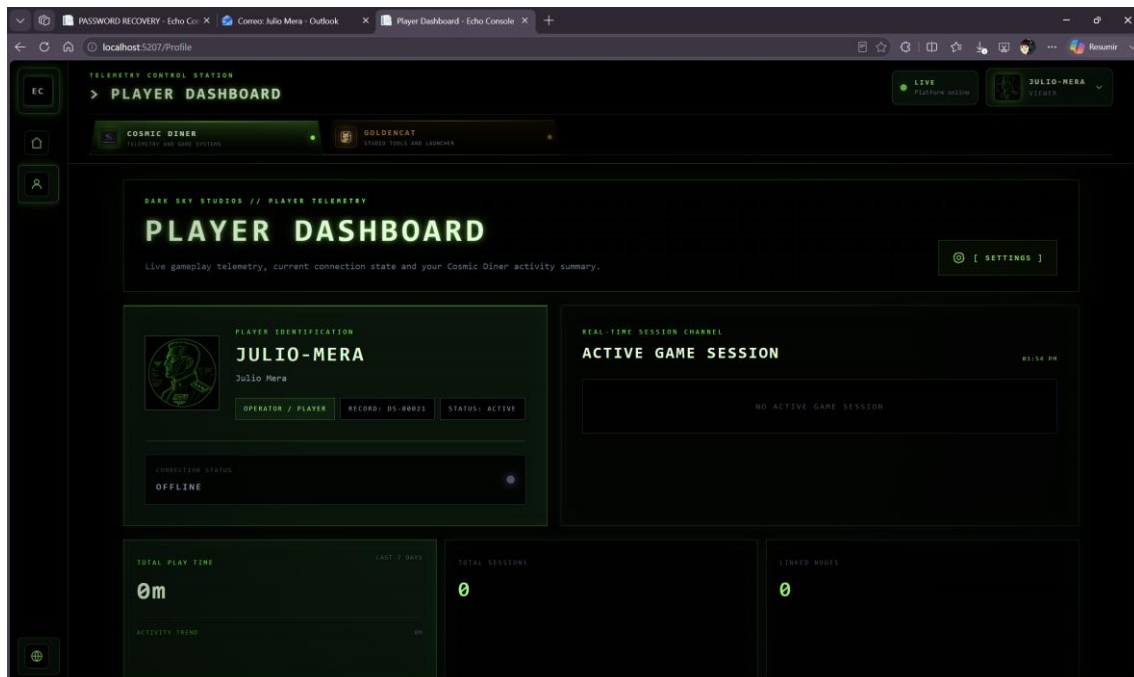
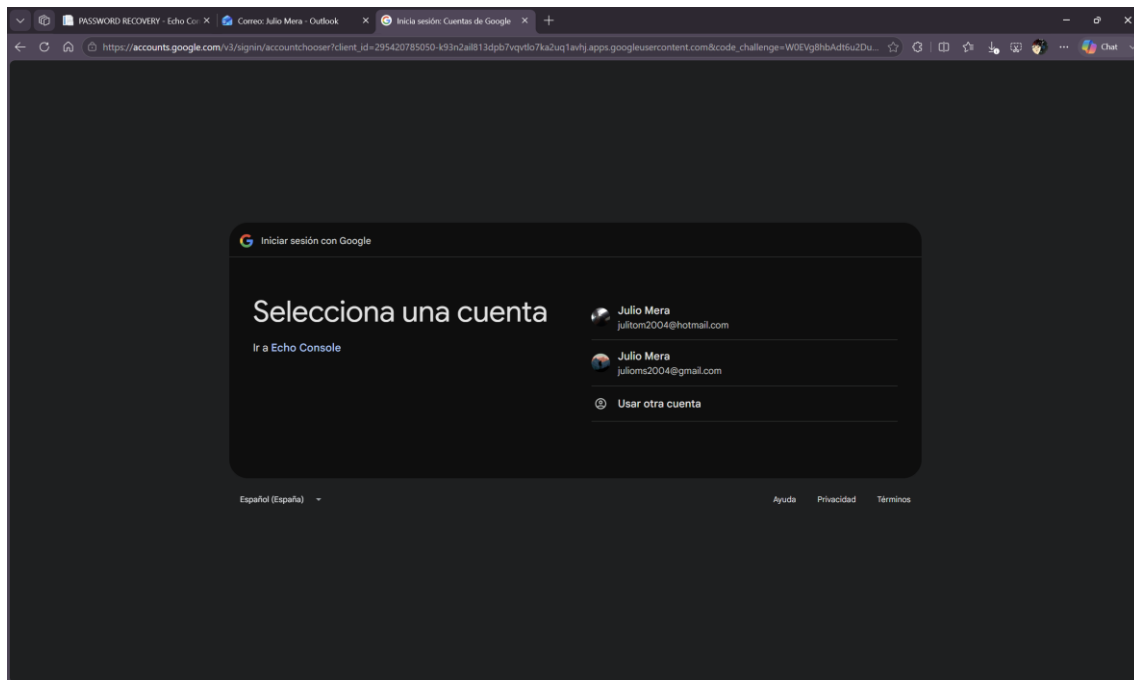


Auth/ForgotPassword: Superficie de identidad con layout dedicado, validación localizada y estética de terminal. Evita exponer el shell administrativo antes de autenticar.

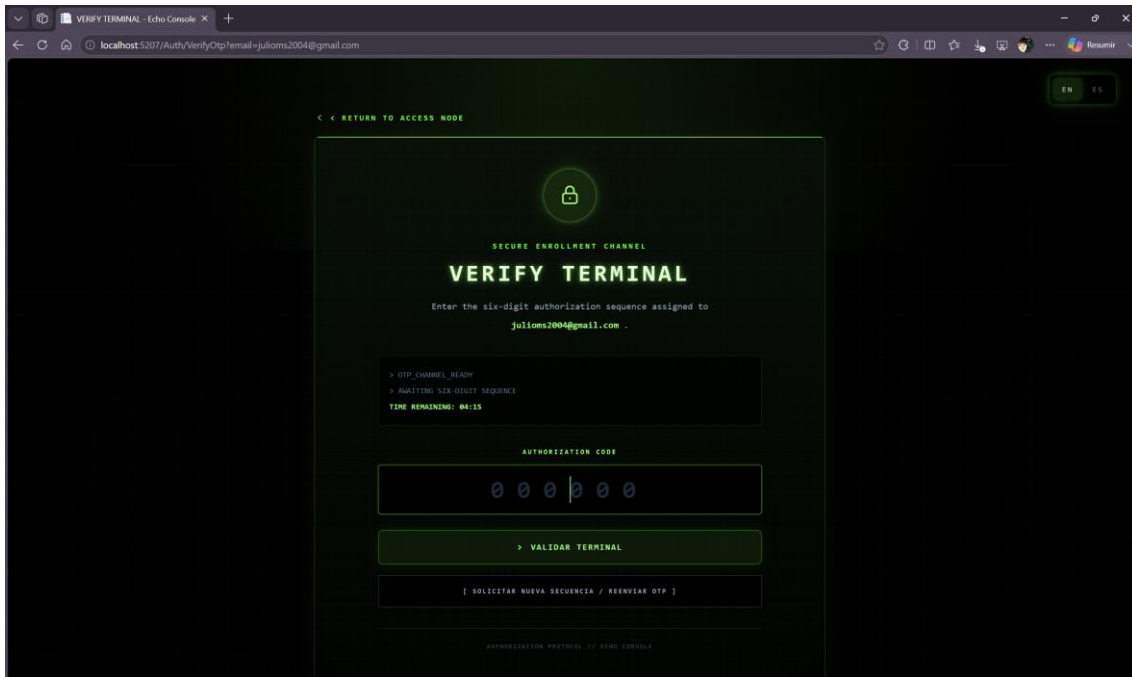
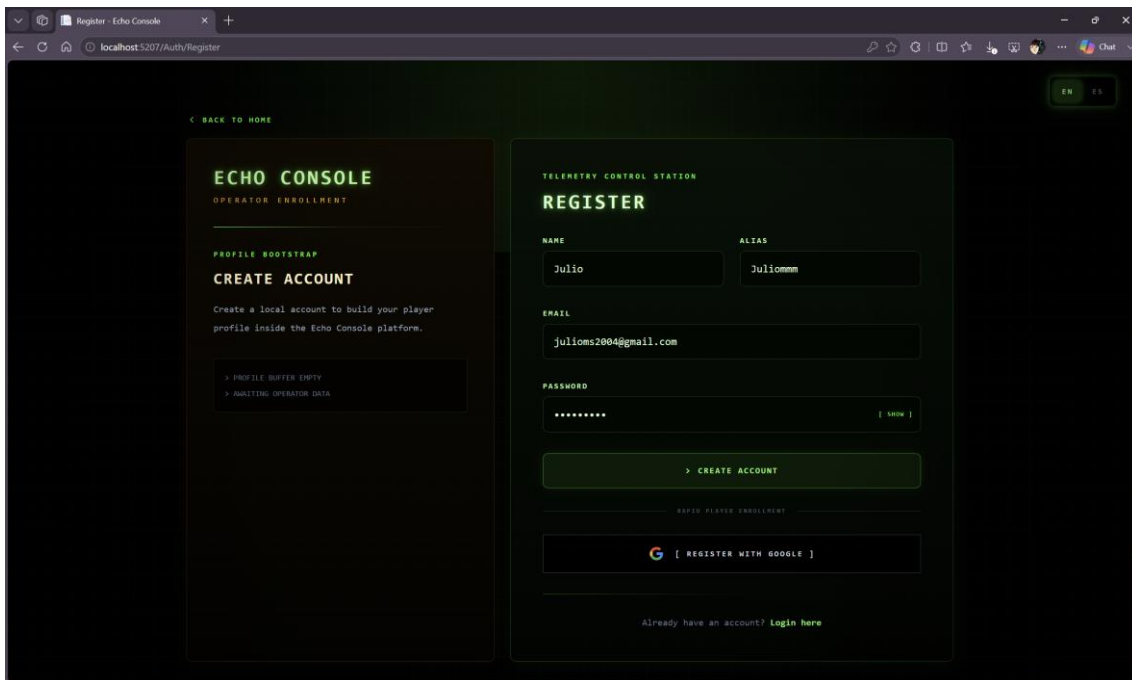


Auth/Login: Superficie de identidad con layout dedicado, validación localizada y estética de terminal. Evita exponer el shell administrativo antes de autenticar.





Auth/Register: Superficie de identidad con layout dedicado, validación localizada y estética de terminal. Evita exponer el shell administrativo antes de autenticar.

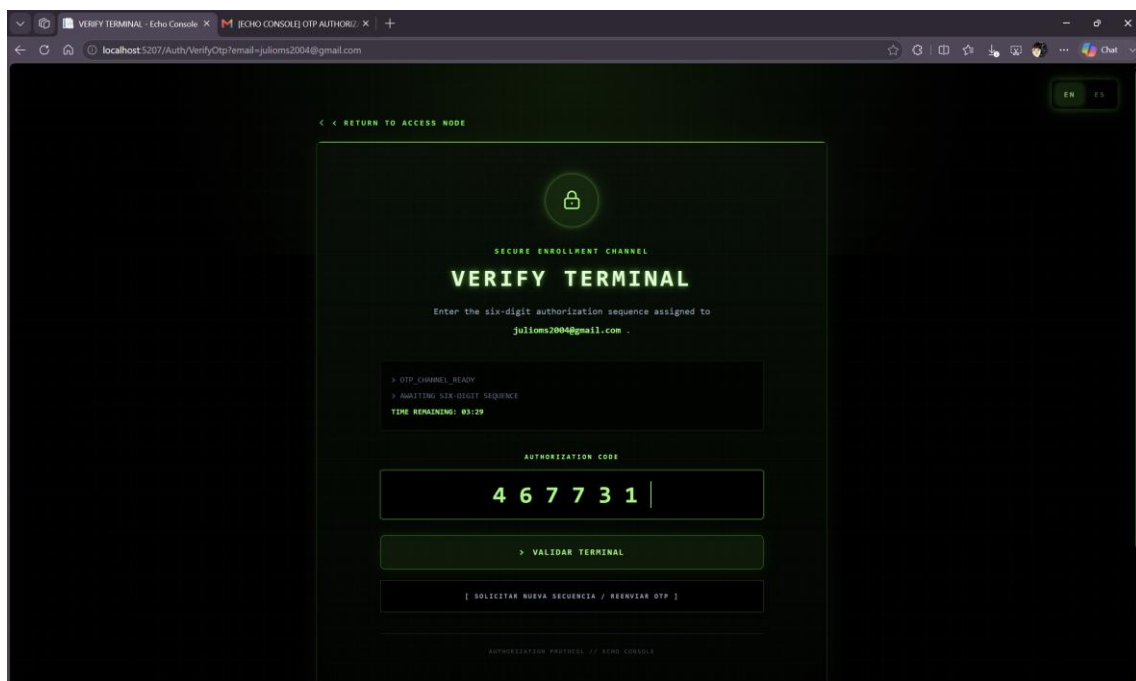
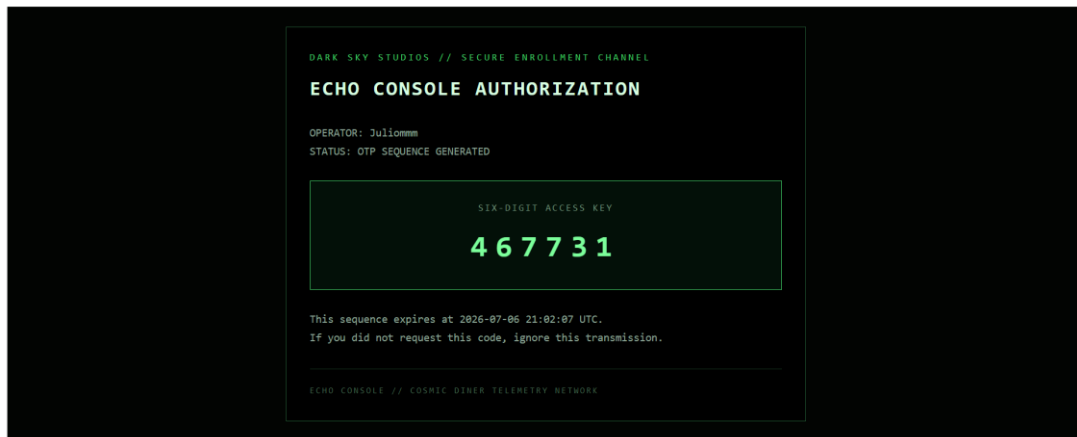


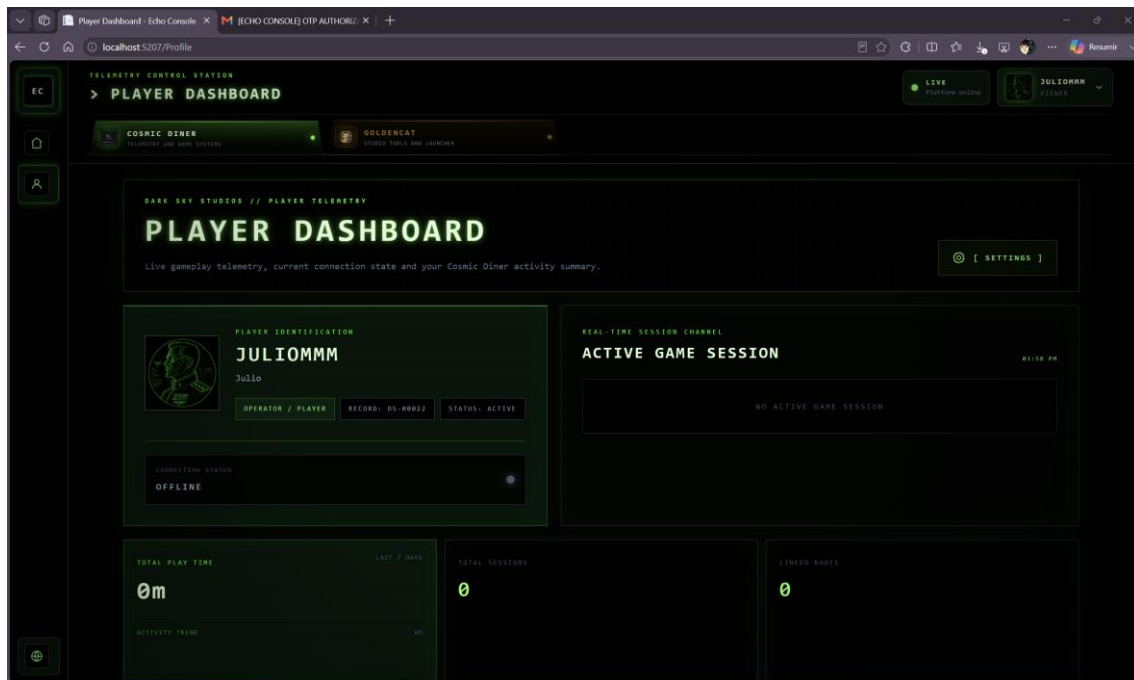
[ECHO CONSOLE] OTP AUTHORIZATION SEQUENCE Recibidos x



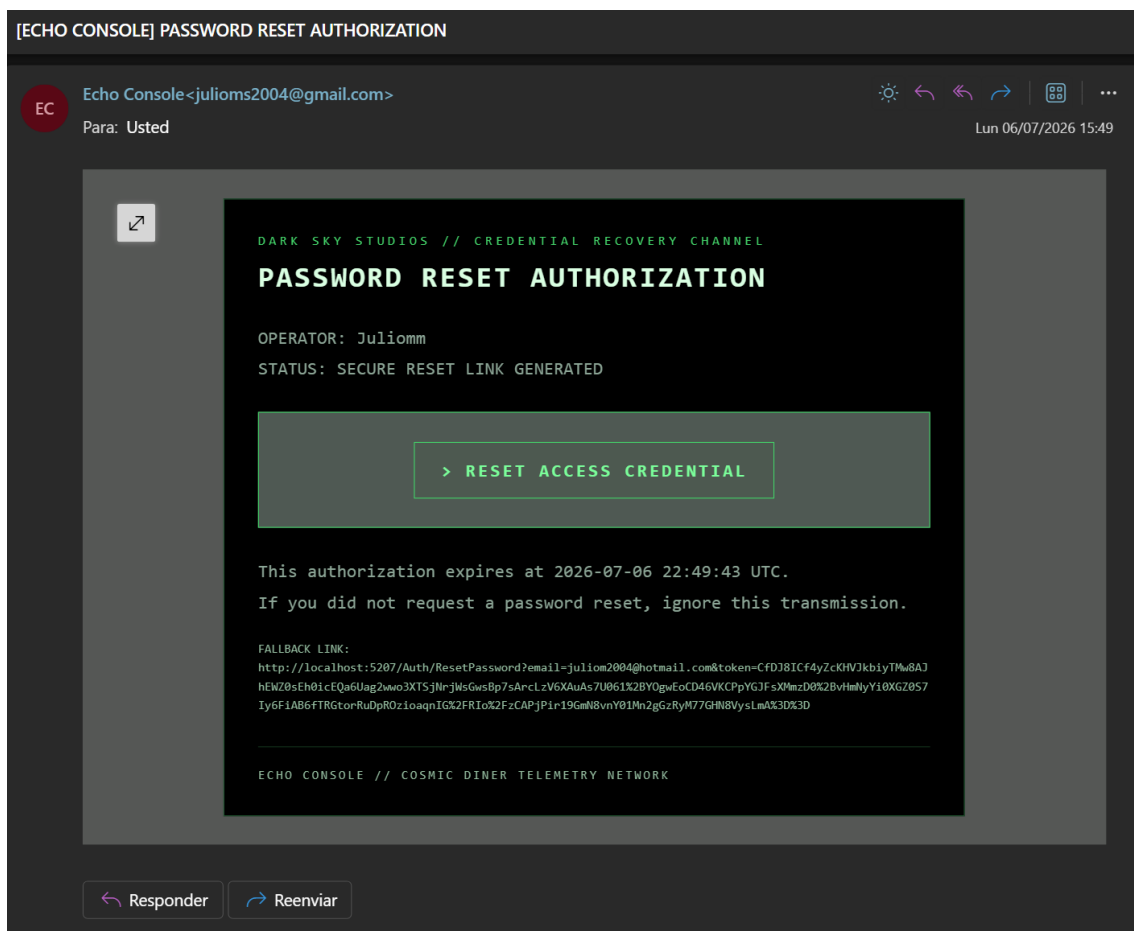
Echo Console <julioms2004@gmail.com>
para mí

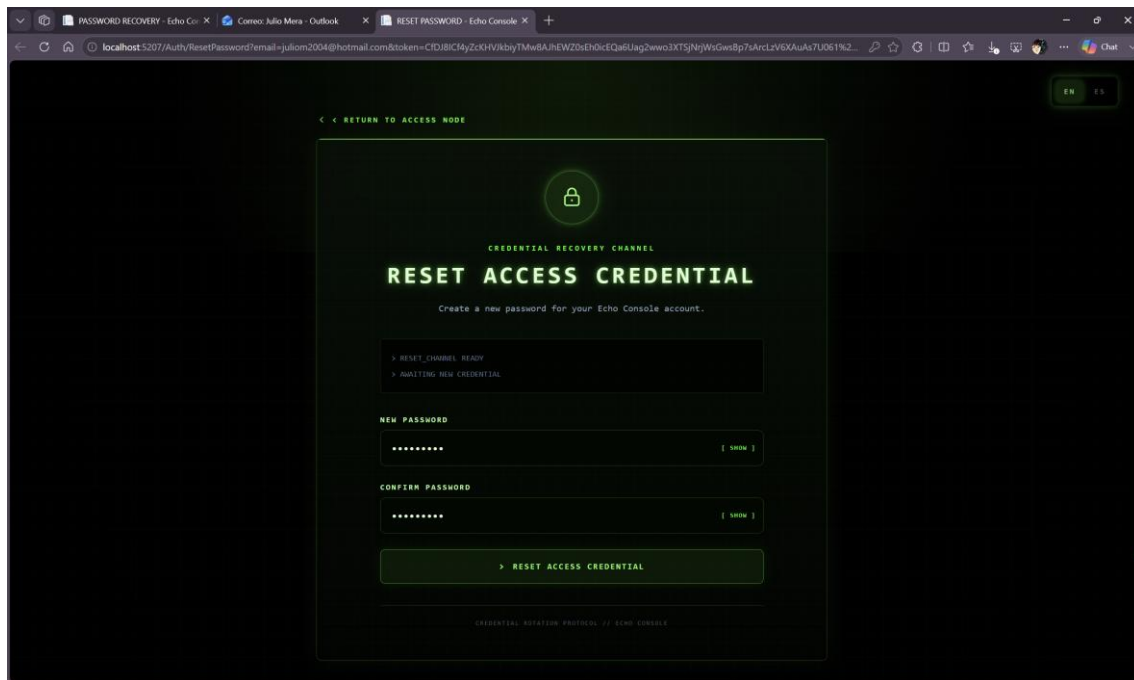
15:57 (hace 1 minuto)





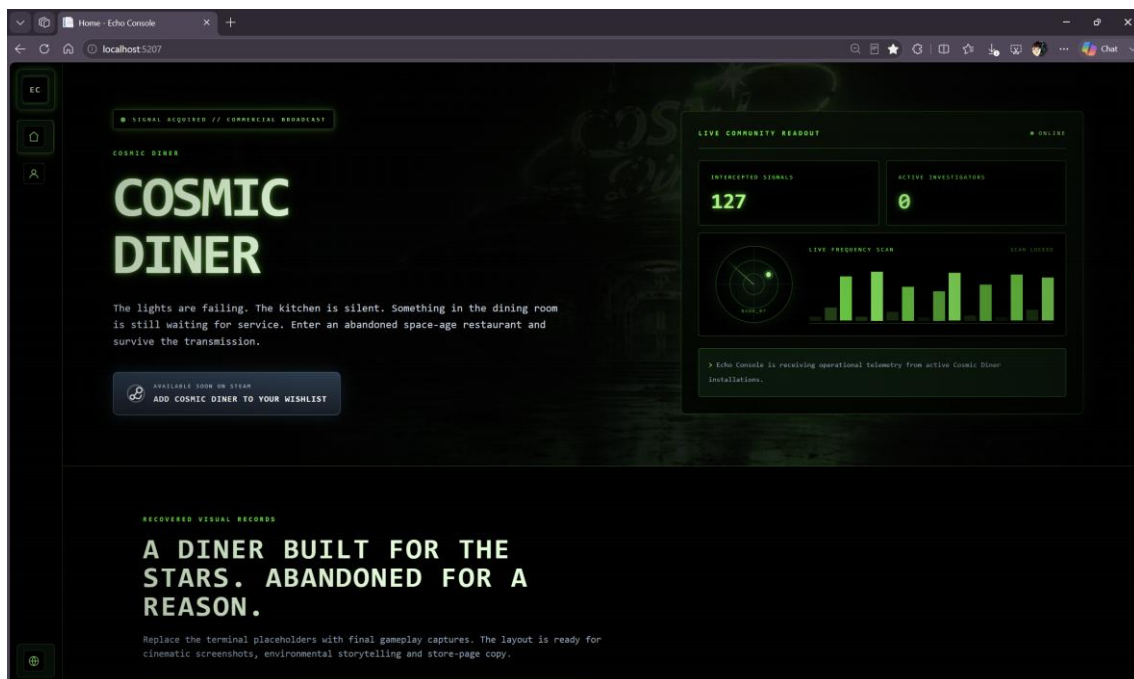
Auth/ResetPassword: Superficie de identidad con layout dedicado, validación localizada y estética de terminal. Evita exponer el shell administrativo antes de autenticar.

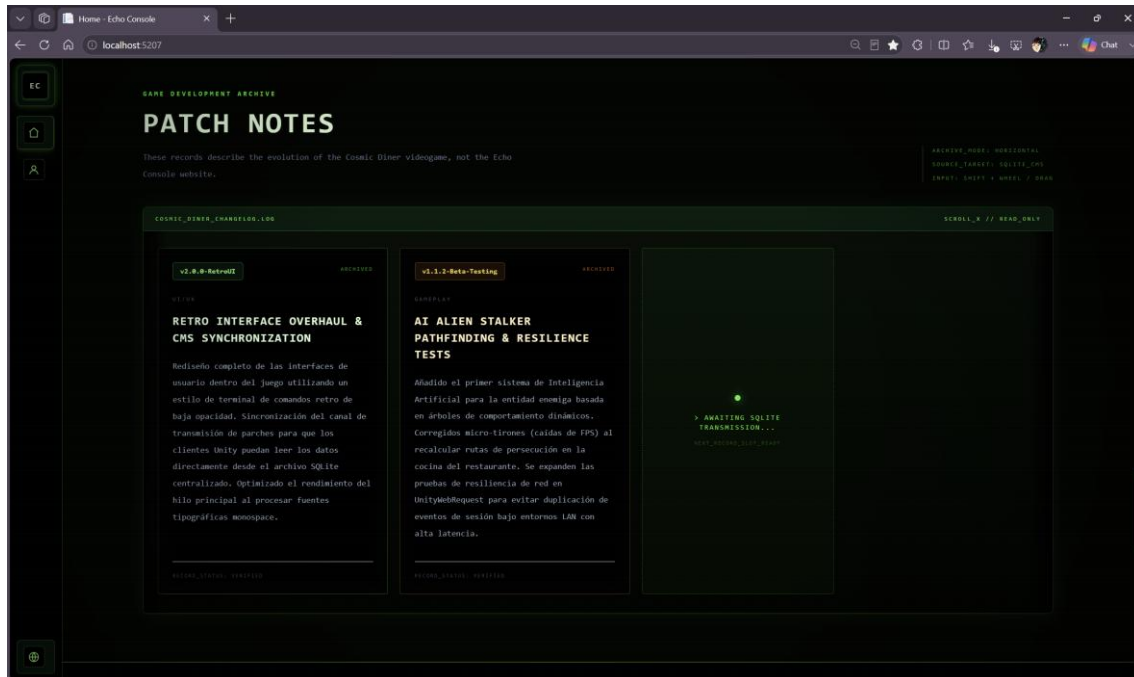
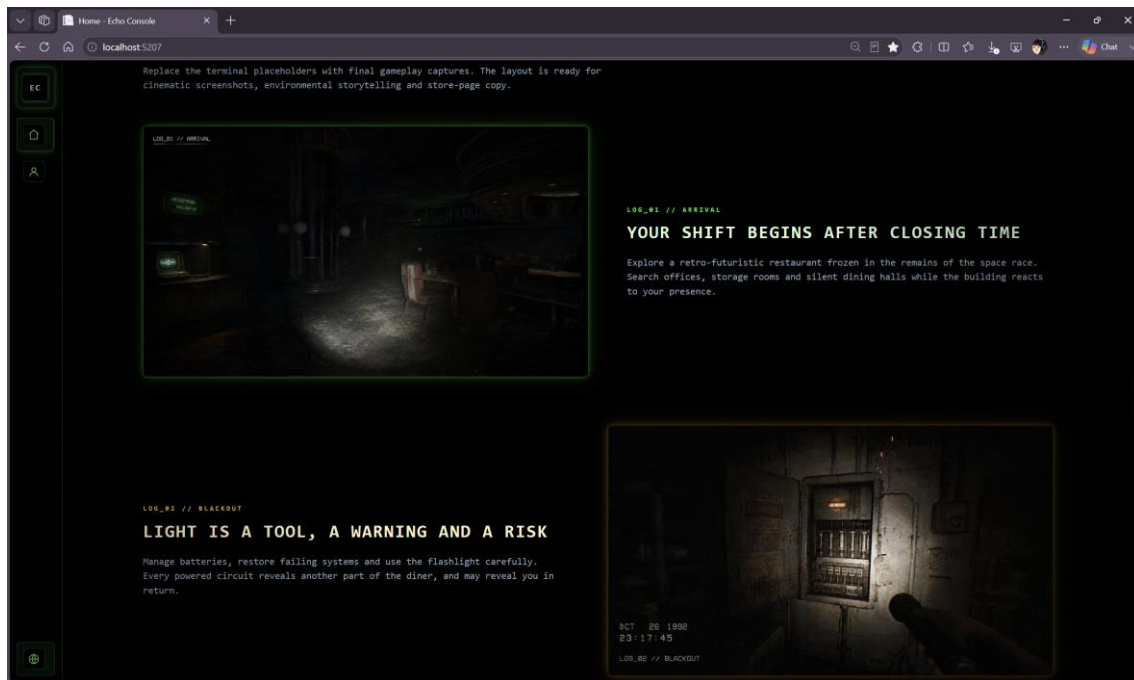


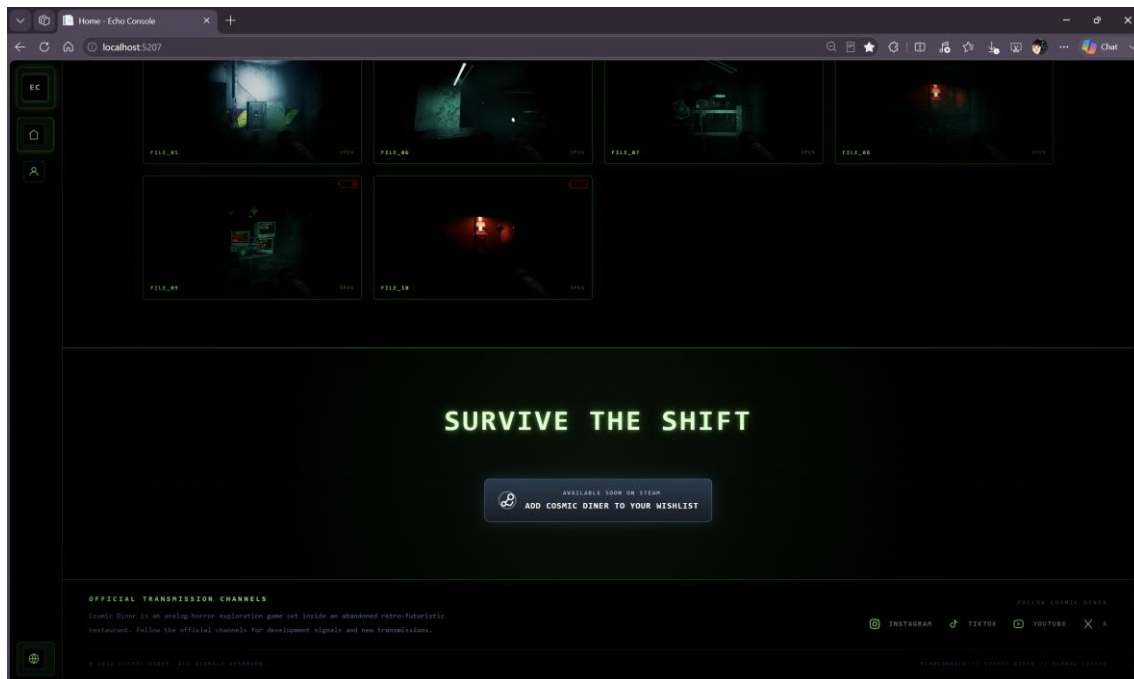
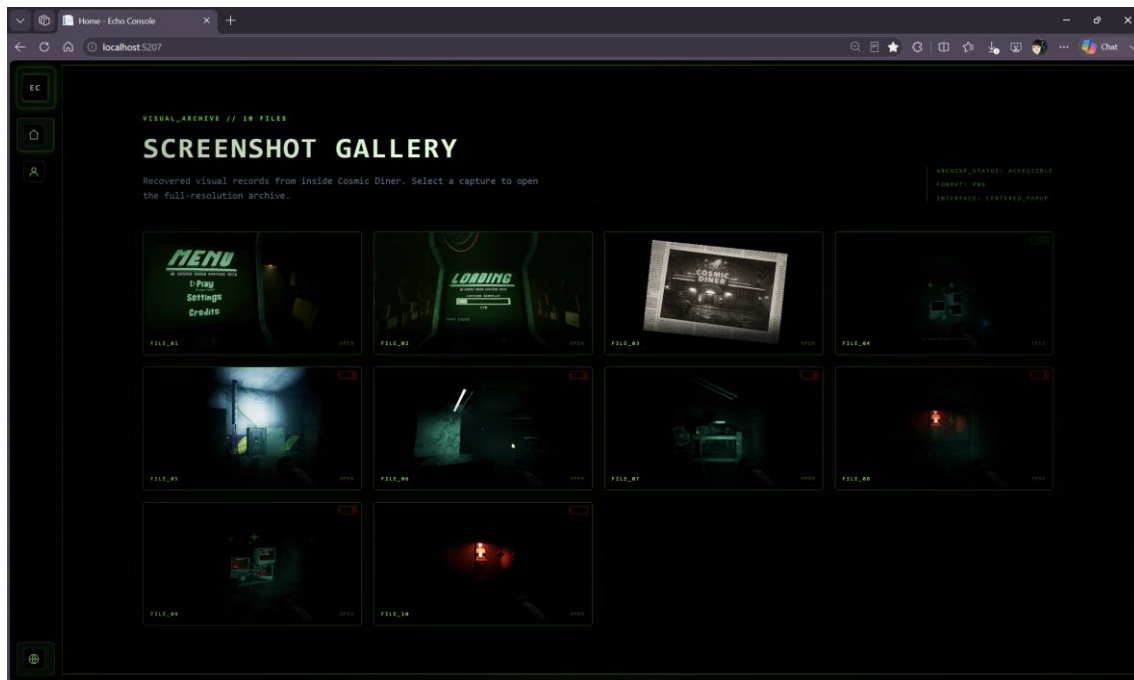


Módulo Home

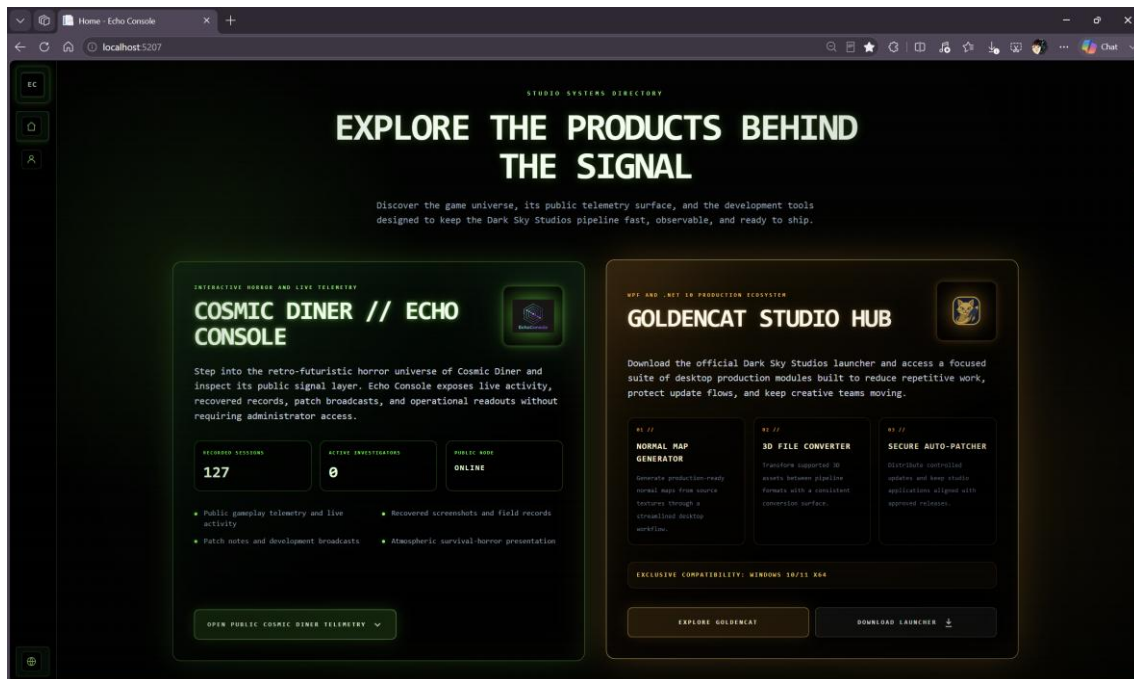
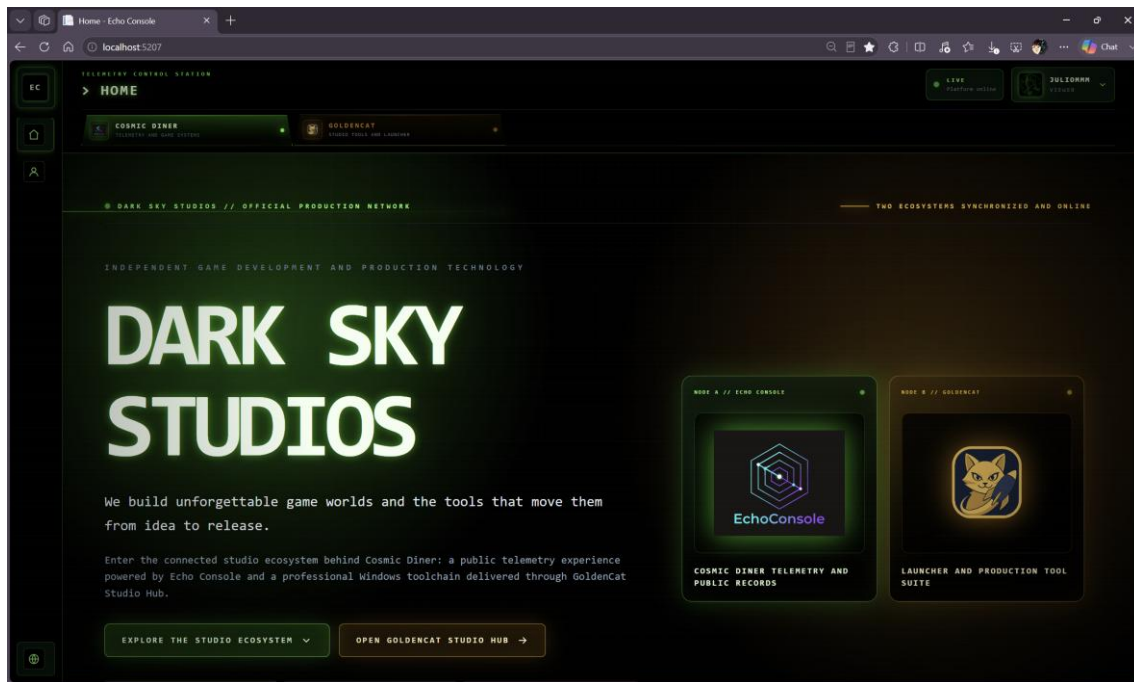
Home/CosmicDiner.cshtml: Página pública de presentación de Cosmic Diner separada de las áreas administrativas.





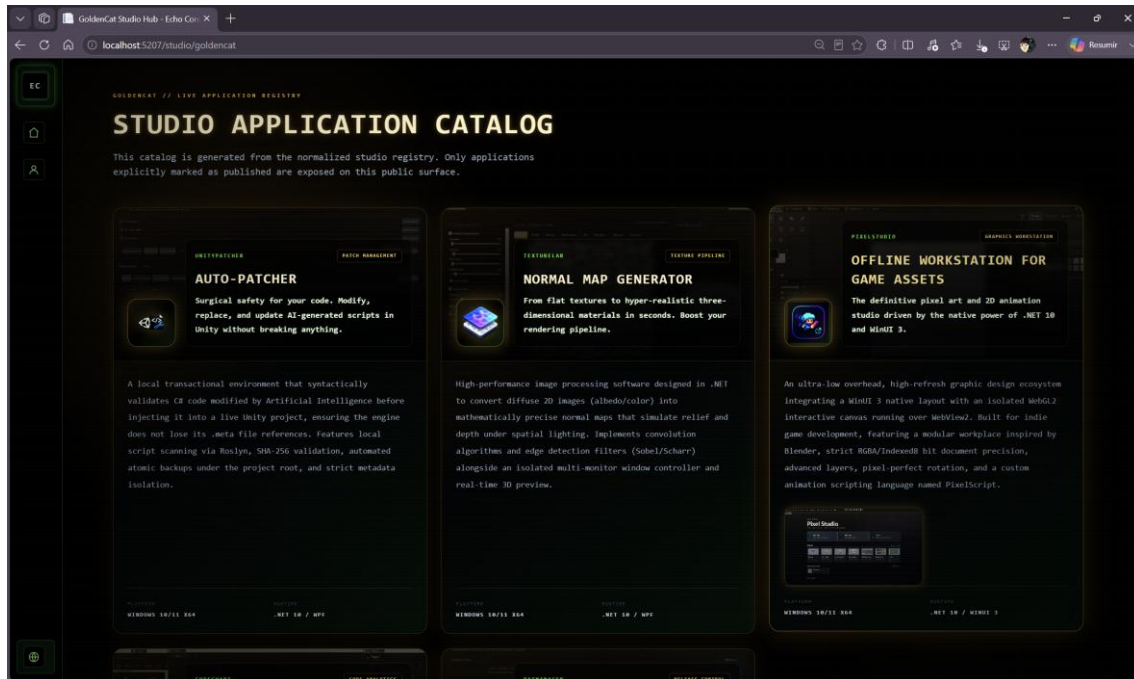
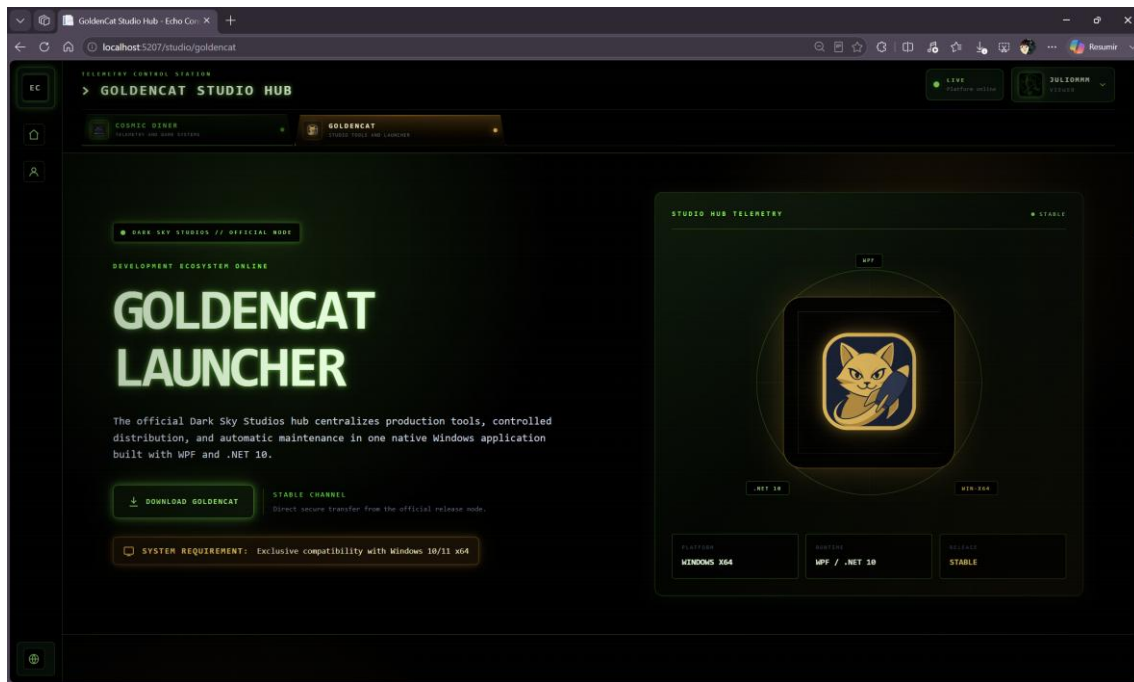


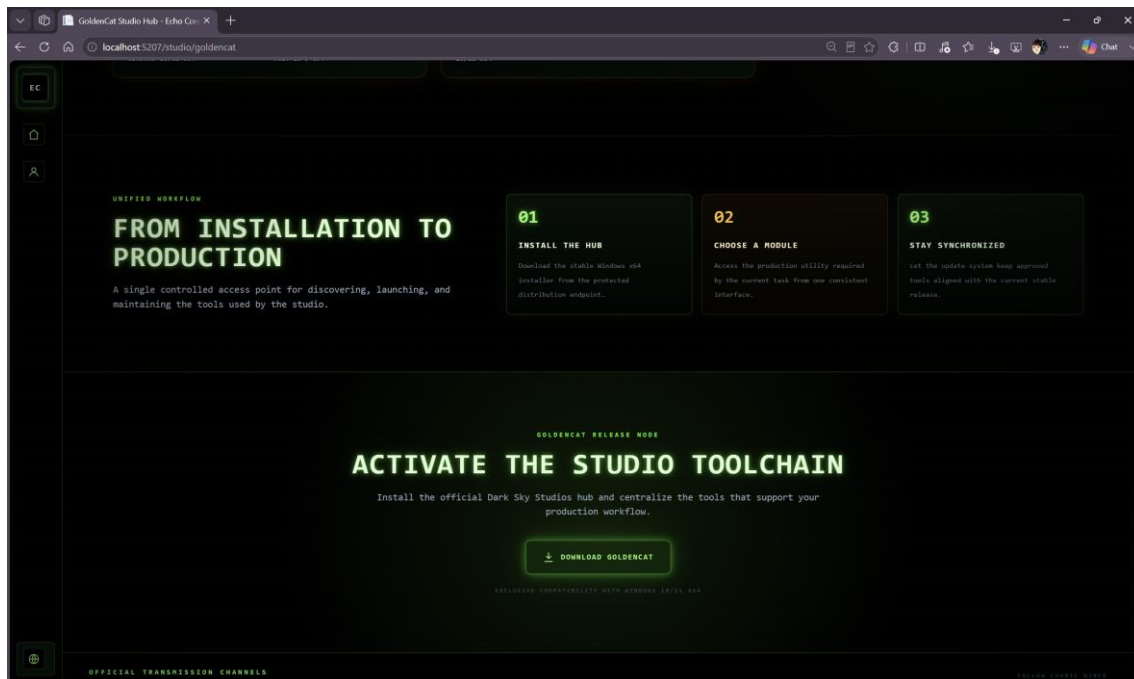
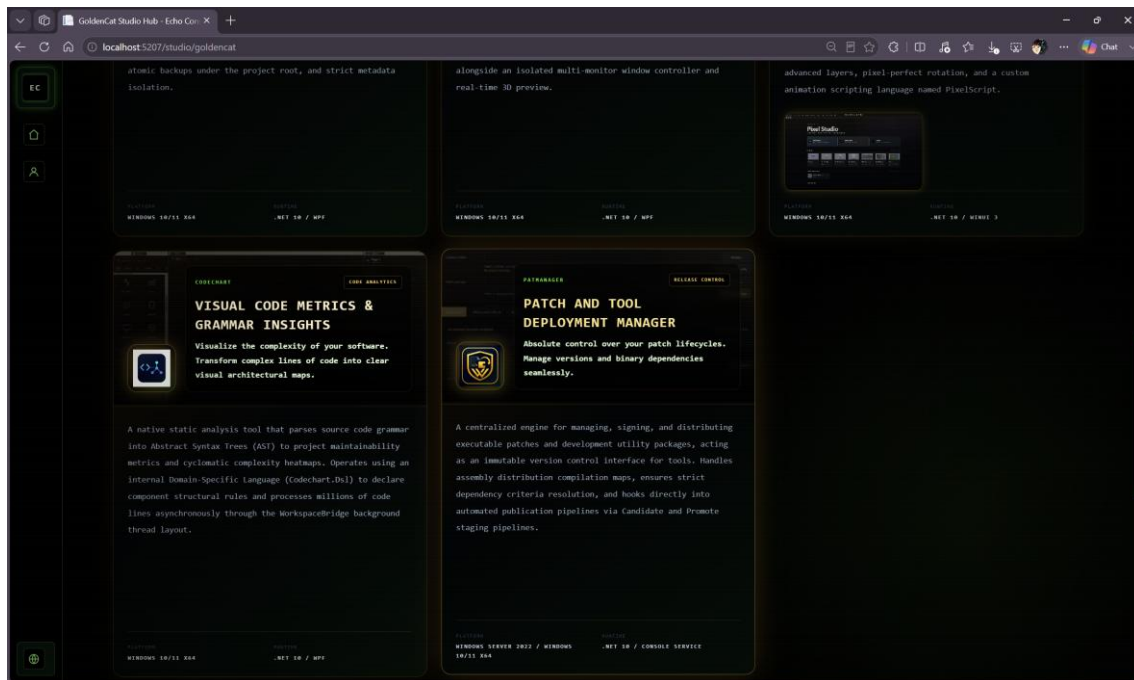
Home/Index.cshtml: Landing principal institucional con telemetría pública, contenido Cosmic Diner, catálogo GoldenCat y wishlist Steam. Incluye integraciones de catálogo, métricas y banner de Steam.



Módulo Ecosystem

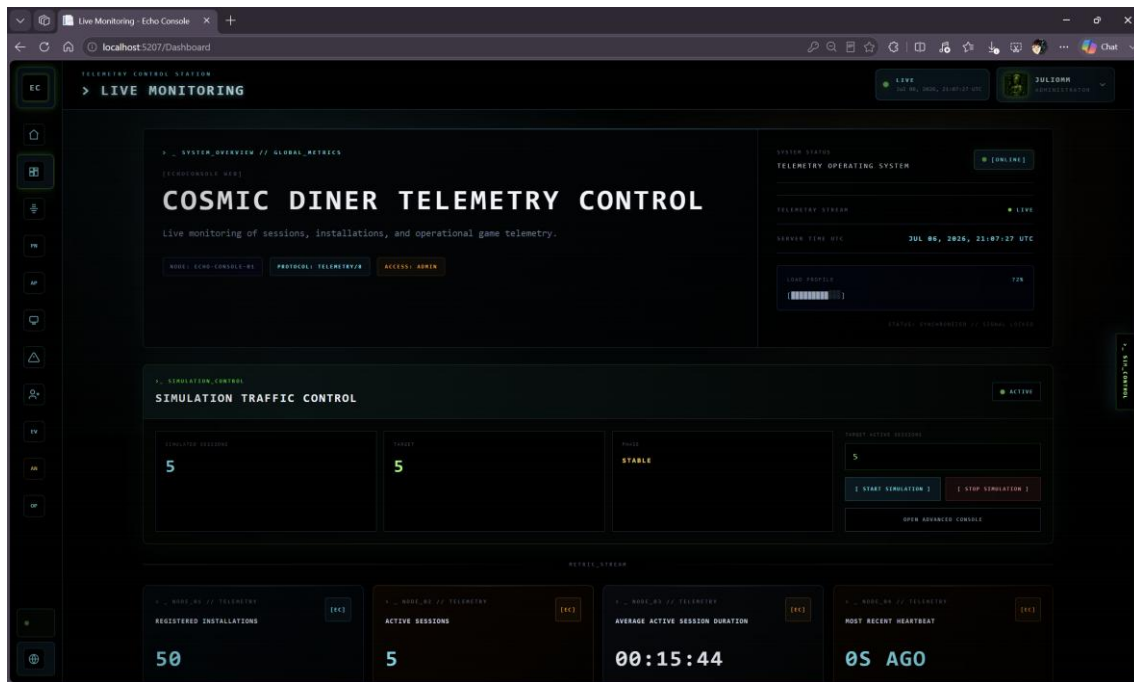
Ecosystem/Index.cshtml: Landing y catálogo GoldenCat. Muestra aplicaciones visibles, galería dinámica y permite descarga segura.



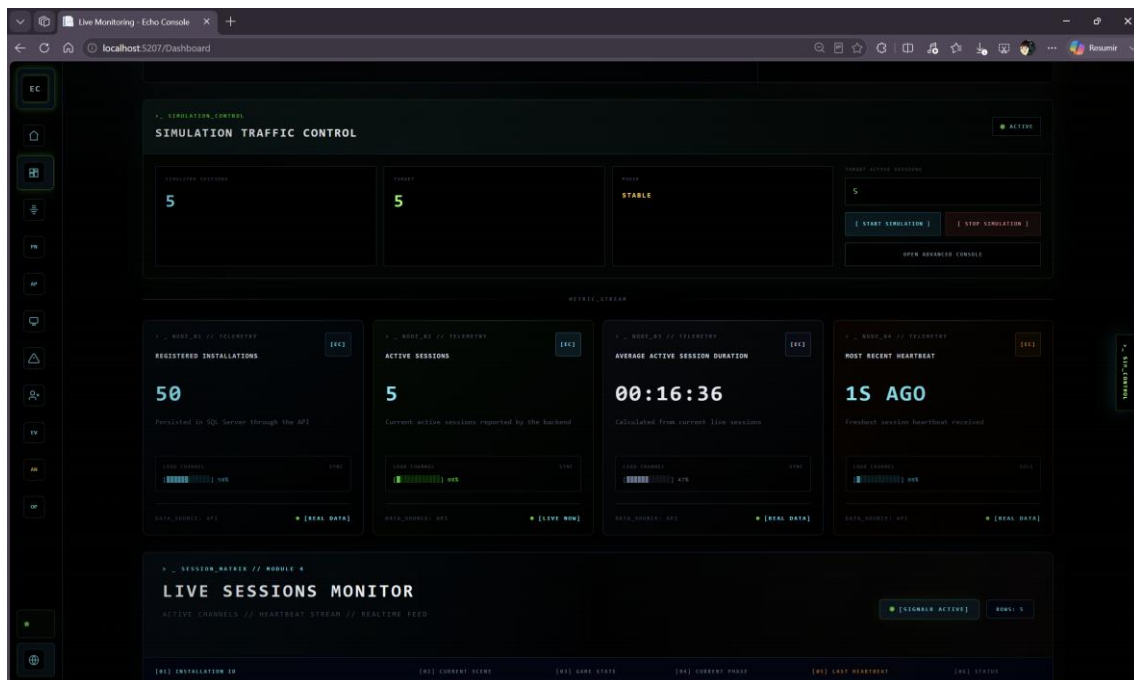


Módulo Dashboard

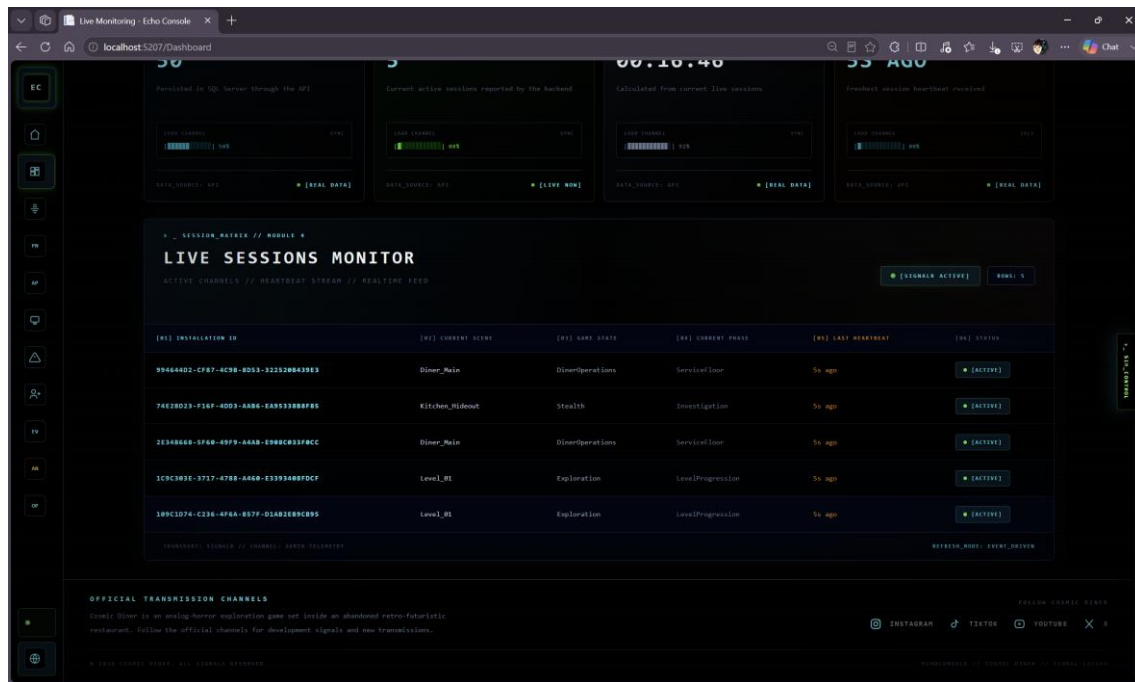
Dashboard/Index.cshtml: Panel operacional de telemetría con KPIs, sesiones activas y actualizaciones en tiempo real mediante SignalR.



Dashboard/_KpiCardsRow.cshtml: Parcial de tarjetas KPI para visualización rápida de métricas del sistema.

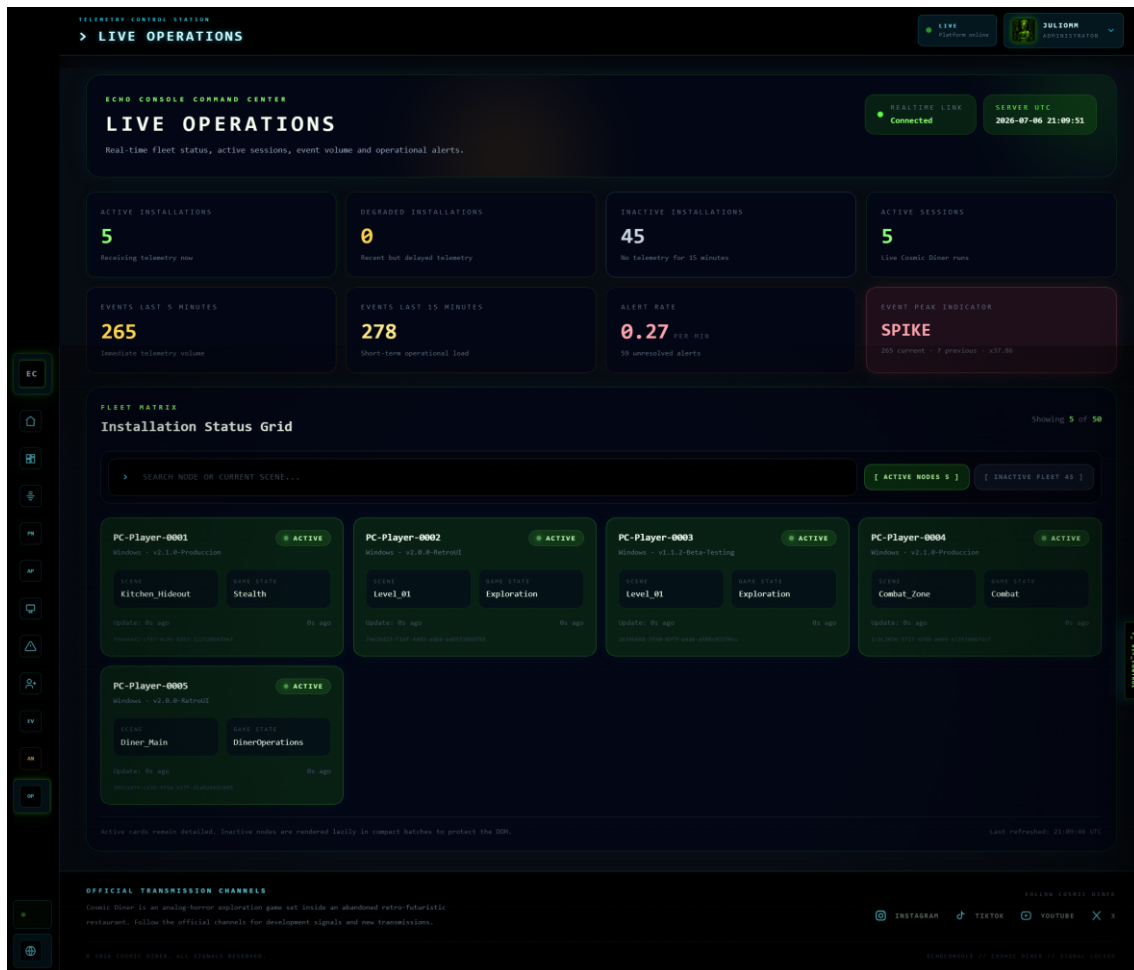


Dashboard/_LiveSessionsTable.cshtml: Tabla de sesiones activas con actualizaciones incrementales en tiempo real.



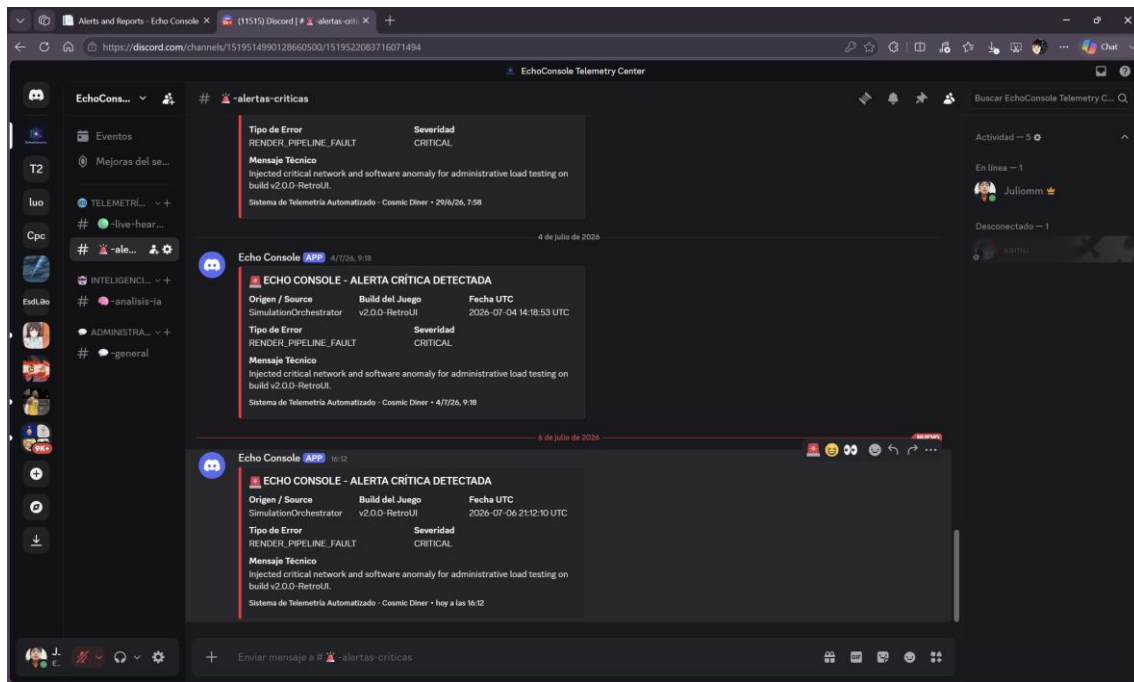
Módulo LiveOperations

LiveOperations/Index.cshtml: Vista de monitoreo en tiempo casi real de flota, sesiones y disponibilidad del sistema.



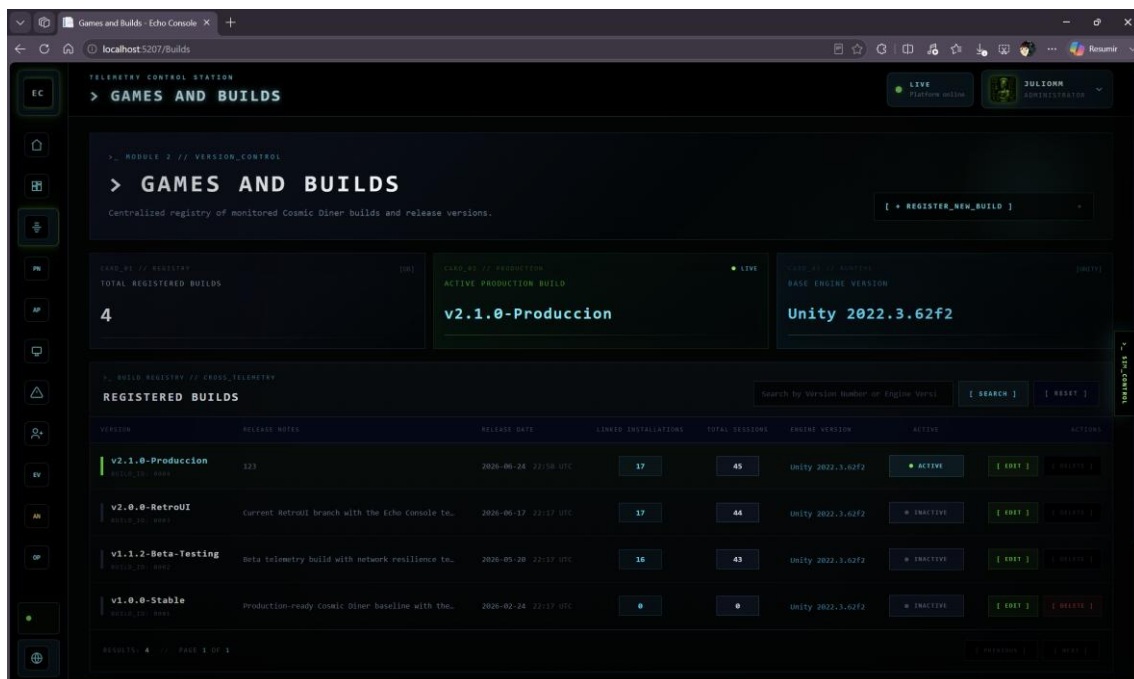
Módulo Alerts

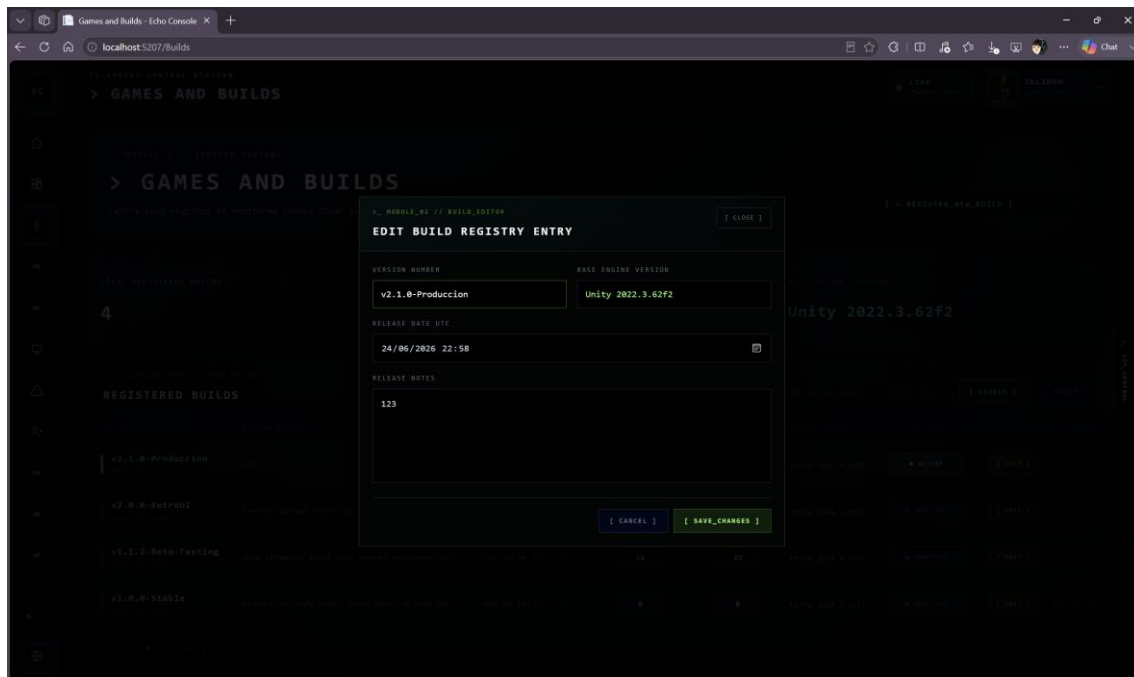
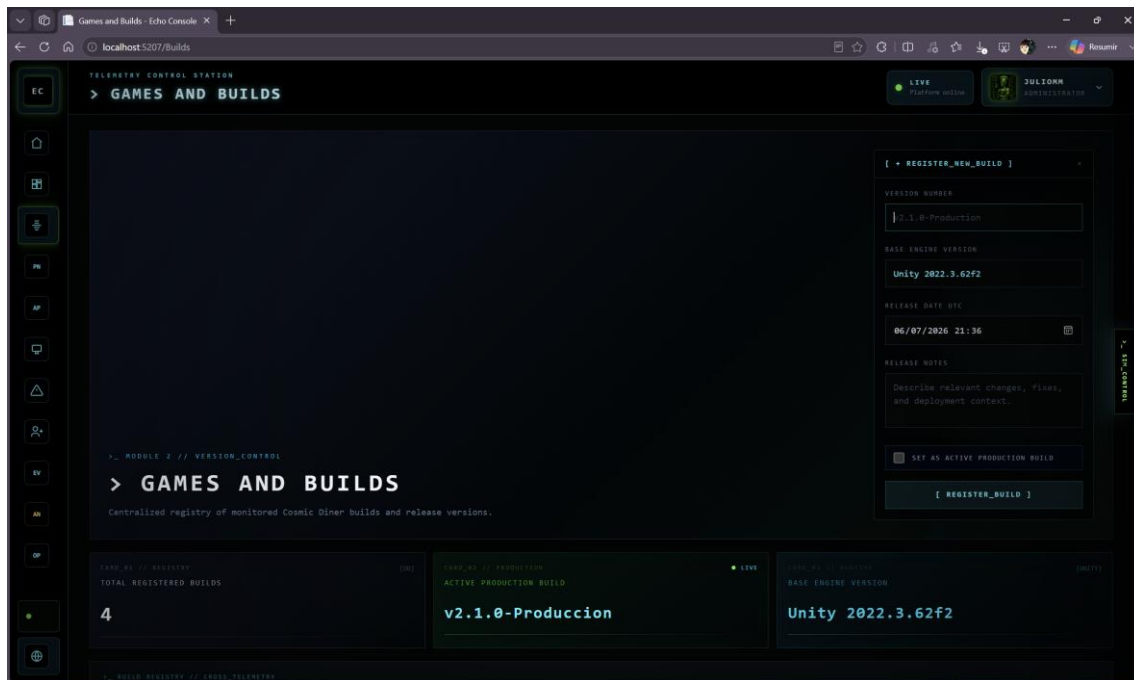
Alerts/Index.cshtml: Centro NOC de alertas con análisis, estados, filtros, tendencias y estado de Discord.



Módulo Builds

Builds/Index.cshtml: Gestión de versiones del juego con control de publicación, activación única y eliminación segura.





Módulo Installations

Installations/Index.cshtml: Inventario de dispositivos con filtros, detalle y gestión administrativa.

Devices and Installations - Echo C X

localhost:5207/installations

TELEMETRY CONTROL STATION

LIVEPlatform onlineJULIOMMADMINISTRATOR

EC

MASTER HARDWARE INVENTORY

Centralized inventory of registered devices and hardware profiles captured from Cosmic Diner installations.

TOTAL DEVICES50

REGISTERED INSTALLATIONS

Search by Installation ID or Device Name

RESULTS: 58

ALIAS / DEVICE	ASSOCIATED BUILD	LICENSE KEYS	ADMIN STATUS	LAST UPDATE	ACTIONS
PC-Player-0001	v2.1.0-Production	Juliommm / #1	ACTIVE	07/06/2026, 21:38:26 UTC	[EDIT ALIAS] [DETAILS] [DELETE]
PC-Player-0002	v2.0.0-RetroUI	ALFA000117 / #20	ACTIVE	07/06/2026, 21:38:26 UTC	[EDIT ALIAS] [DETAILS] [DELETE]
PC-Player-0003	v1.1.2-Beta-Testing	Juliommm / #1	ACTIVE	07/06/2026, 21:38:26 UTC	[EDIT ALIAS] [DETAILS] [DELETE]
PC-Player-0004	v2.1.0-Production	Juliommm / #1	ACTIVE	07/06/2026, 21:38:26 UTC	[EDIT ALIAS] [DETAILS] [DELETE]
PC-Player-0005	v2.0.0-RetroUI	Juliommm / #22	ACTIVE	07/06/2026, 21:38:26 UTC	[EDIT ALIAS] [DETAILS] [DELETE]
PC-Player-0006	v1.1.2-Beta-Testing	UNASSIGNED	ACTIVE	07/06/2026, 14:21:00 UTC	[EDIT ALIAS] [DETAILS] [DELETE]

Devices and Installations - Echo C X

localhost:5207/installations

TELEMETRY CONTROL STATION

LIVEPlatform onlineJULIOMMADMINISTRATOR

EC

MASTER HARDWARE INVENTORY

Centralized inventory of registered devices and hardware profiles captured from Cosmic Diner installations.

TOTAL DEVICES50

REGISTERED INSTALLATIONS

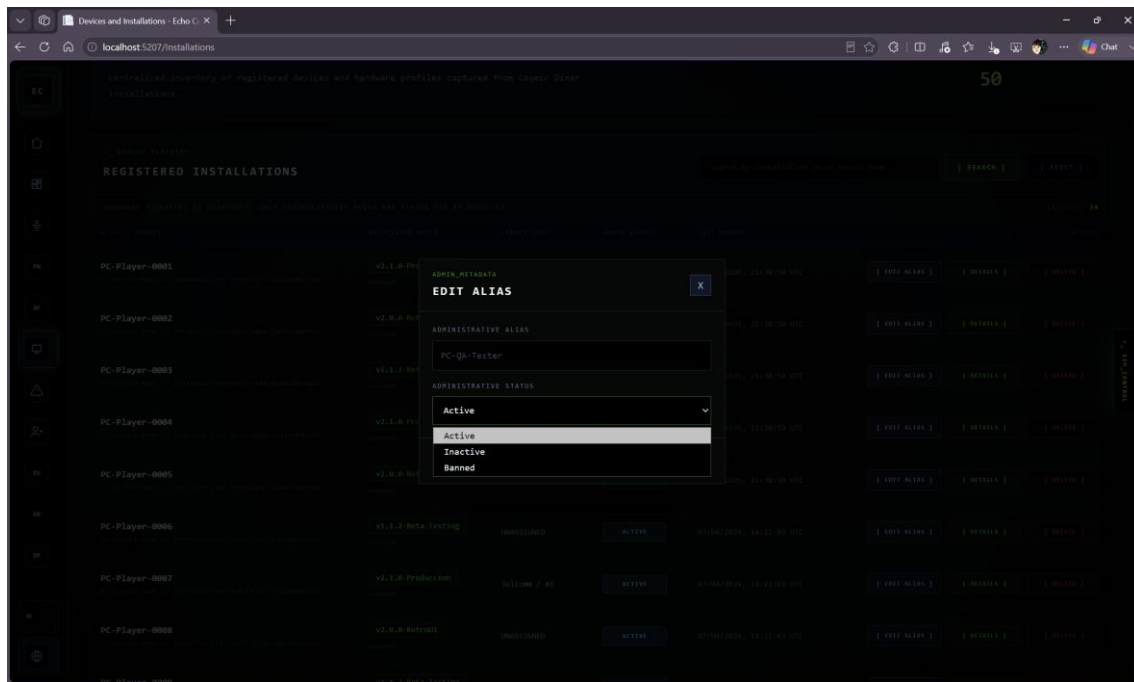
Search by Installation ID or Device Name

RESULTS: 58

READ_ONLY_HARDWAREDETAILS

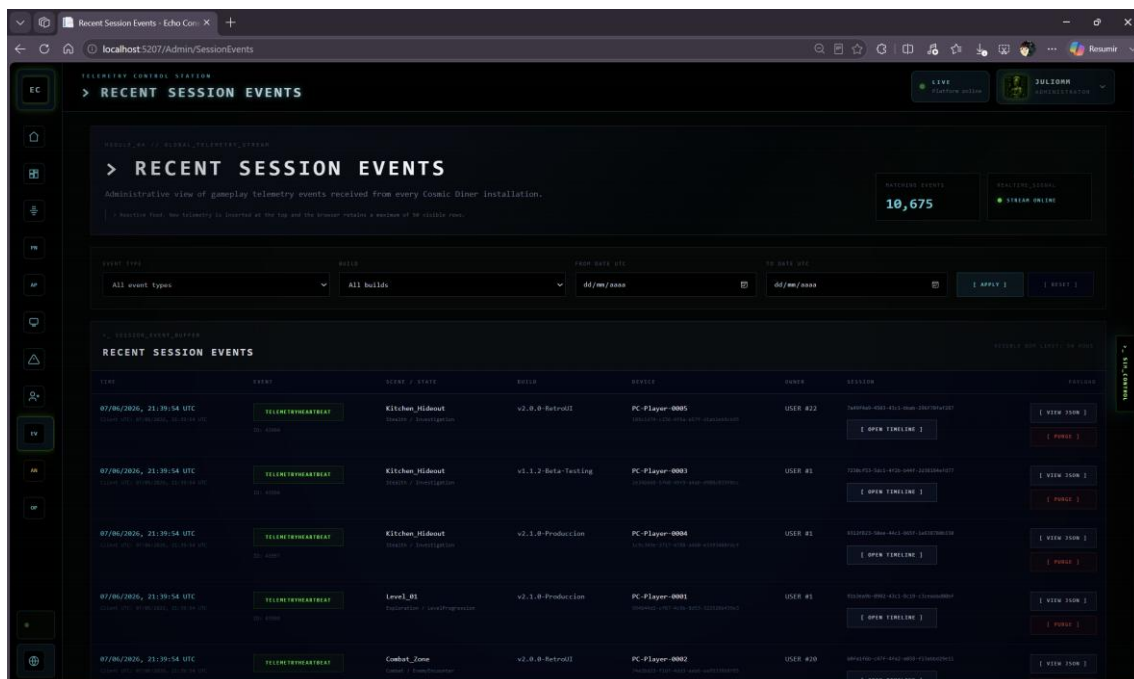
INSTALLATION_ID	99a644d2-cf87-4c9b-8d53-322520b439e3
ADMIN_ALIAS	-
ADMIN_STATUS	Active
NAME_CODE	cosmic-diner
PLATFORM	Windows
DEVICE_MODEL	Echo Simulation Mode
PROCESSOR	Virtual CPU
RAM	16,384 MB
FIRST_UPDATE_DAT	06/30/2026, 19:11:13 UTC
LAST_UPDATE_DAT	07/06/2026, 21:38:46 UTC

[CLOSE]

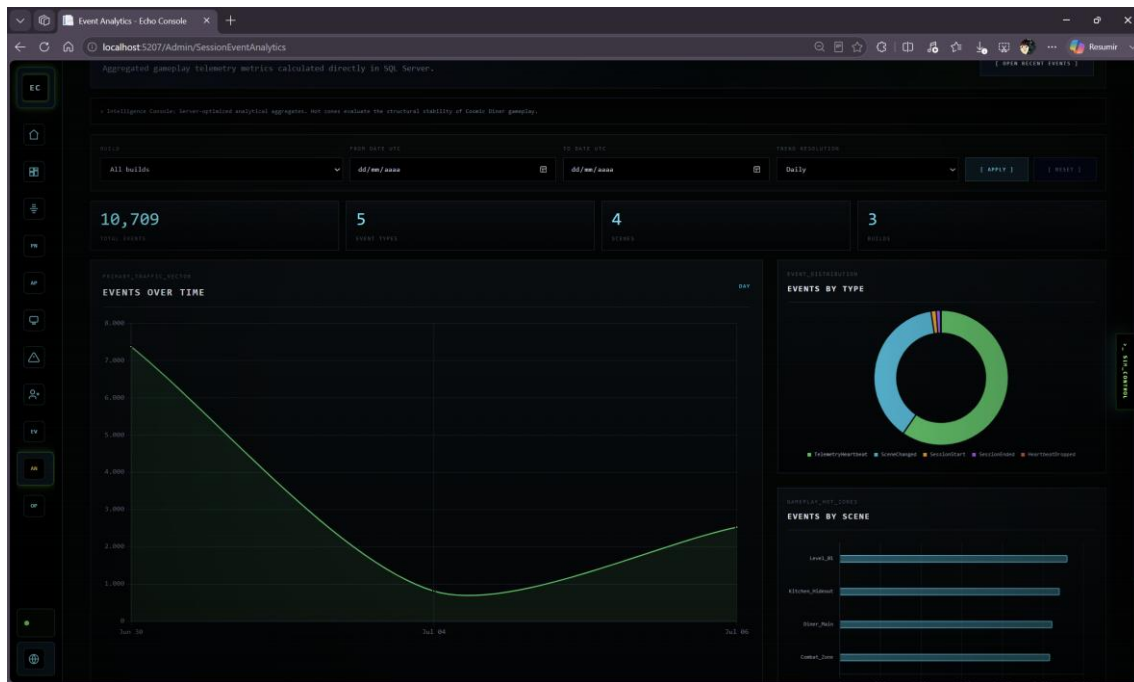


Módulo SessionEvents

SessionEvents/Details.cshtml: Explorador detallado de eventos de juego con timeline y purga controlada.



SessionEvents/Index.cshtml: Listado de eventos de sesión con filtros, paginación y operaciones administrativas.



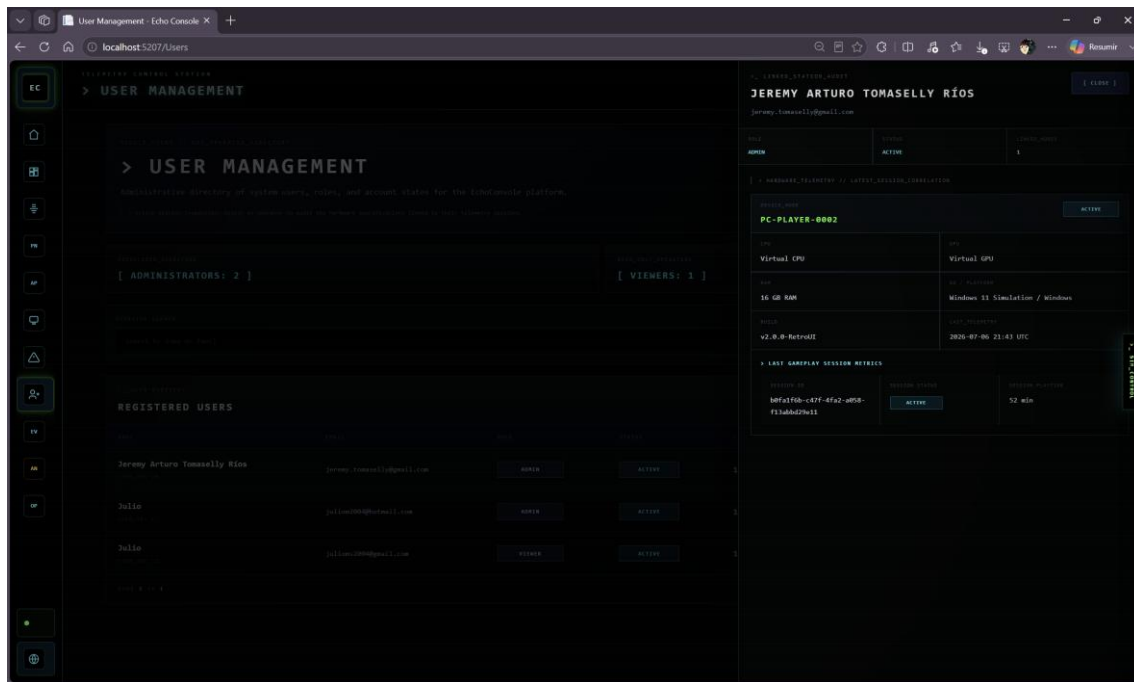
Módulo Users

Users/Index.cshtml: Directorio administrativo de usuarios, roles, estados y sesiones asociadas.

The screenshot displays the 'User Management - Echo Console' interface. The main heading is 'USER MANAGEMENT'. Below this, there are two summary cards: 'ADMINISTRATORS: 2' and 'VIEWERS: 1'. A search bar is present with the text 'Search by Name or Email'. The main section is titled 'REGISTERED USERS' and contains a table with the following data:

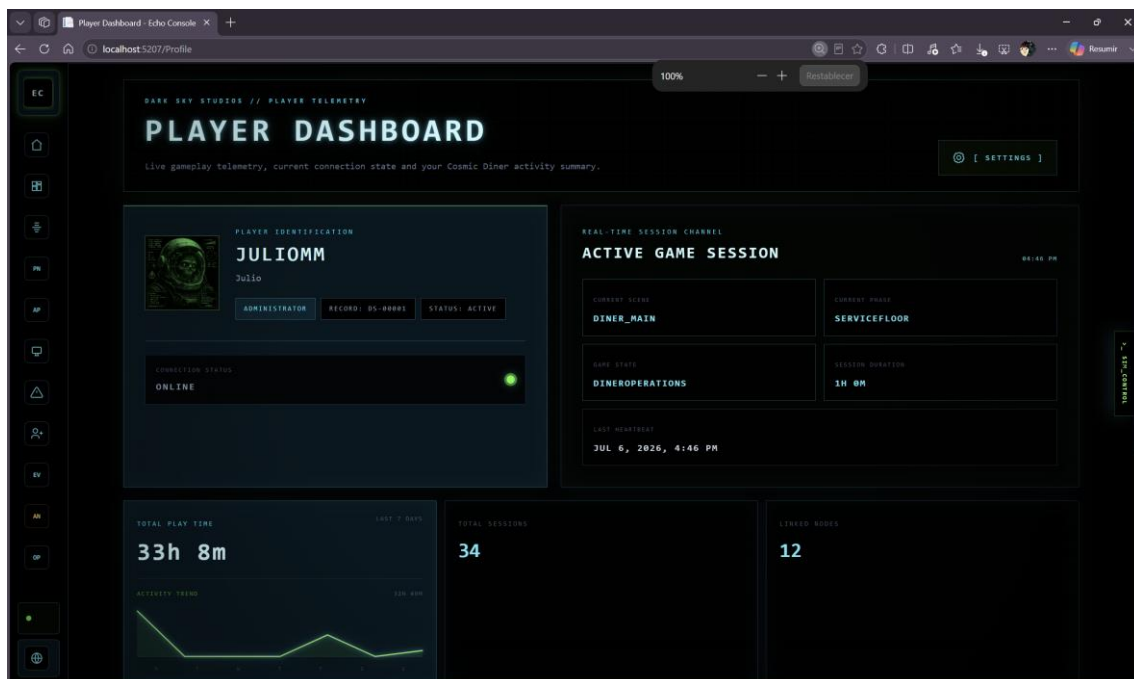
NAME	EMAIL	ROLE	STATUS	SESSIONS	LAST FOLLOWUP	ACTIONS
Jeremy Arturo Tomasselly Rios	jeremy.tomasselly@gmail.com	ADMIN	ACTIVE	1	2020-07-06 21:43 UTC	[RESET]
Julio	julio2004@hotmail.com	ADMIN	ACTIVE	12	2020-07-06 21:43 UTC	[RESET]
Julio	julio2004@gmail.com	VIEWER	ACTIVE	1	2020-07-06 21:43 UTC	[RESET]

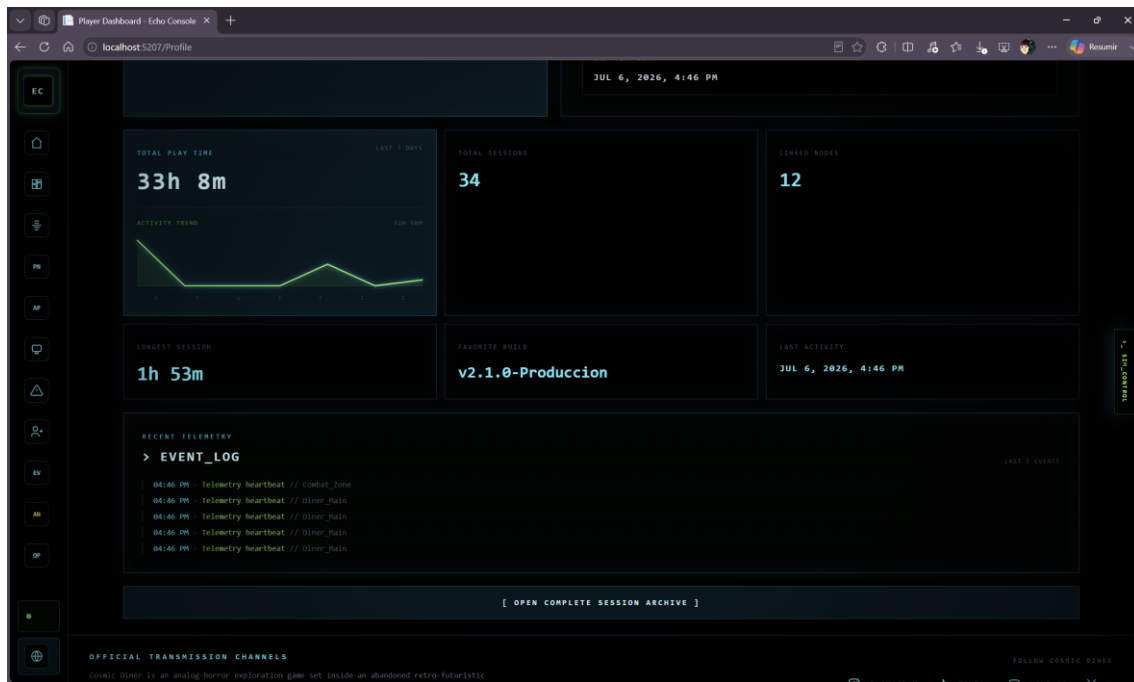
At the bottom of the table, it says 'PAGE 1 OF 1'.



Módulo Profile

Profile/Index.cshhtml: Panel de autoservicio del usuario con identidad, sesiones, seguridad y preferencias.



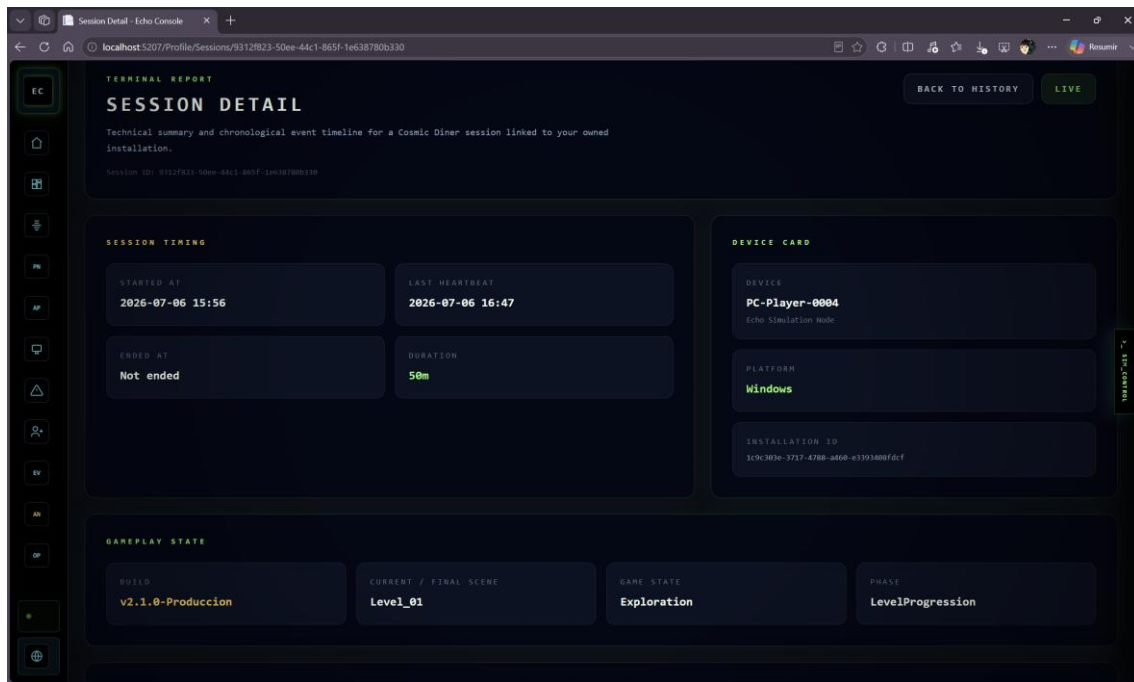


Profile/SessionDetail.cshtml: Detalle de sesión del usuario con historial y eventos asociados.

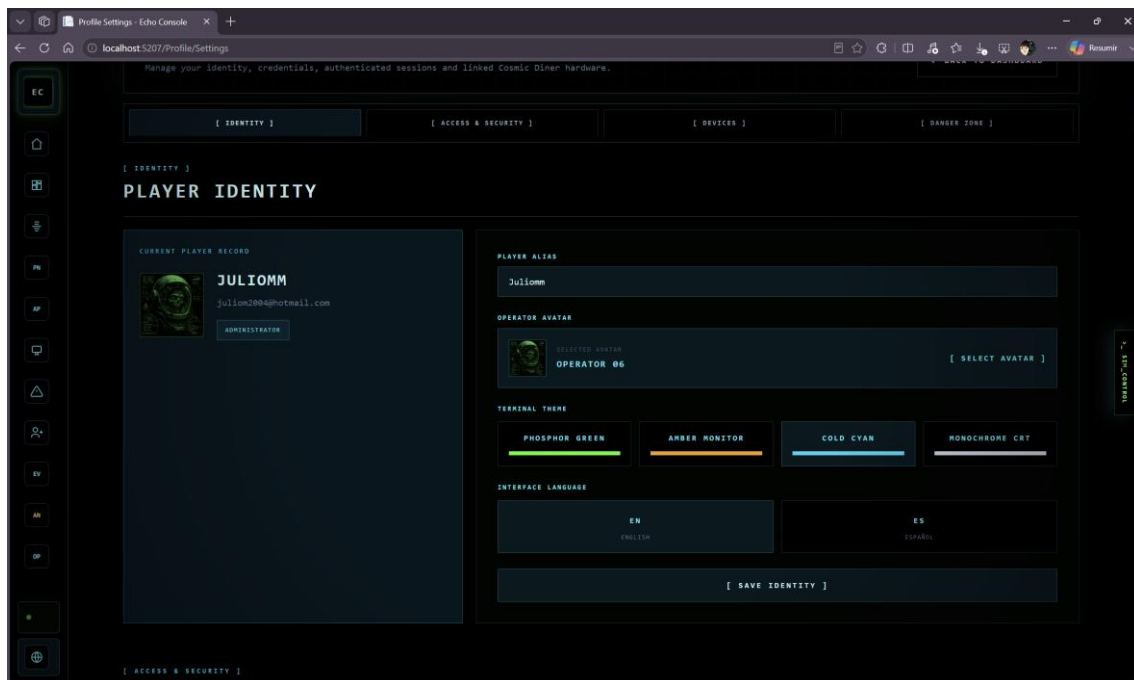
The screenshot displays the 'Session History - Echo Console' interface. The main content area is titled 'COSMIC DINER TELEMETRY PERSONAL SESSION HISTORY'. It includes a 'BACK TO PROFILE' button and a '34 session(s)' indicator. Below the title, there is a table of session history:

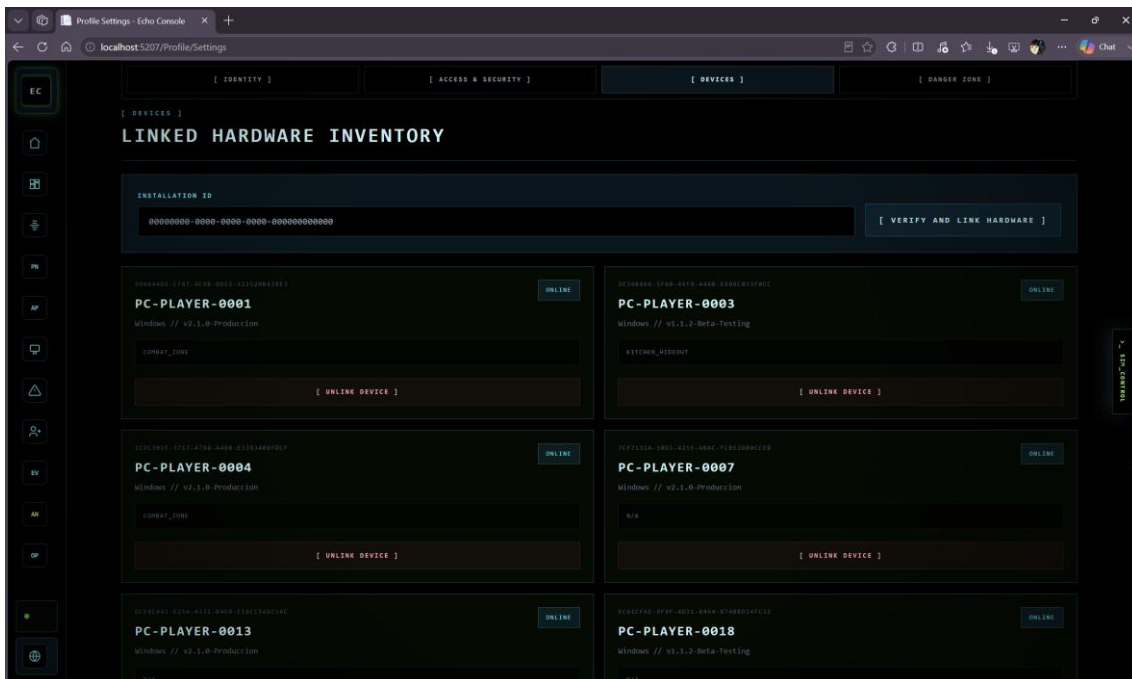
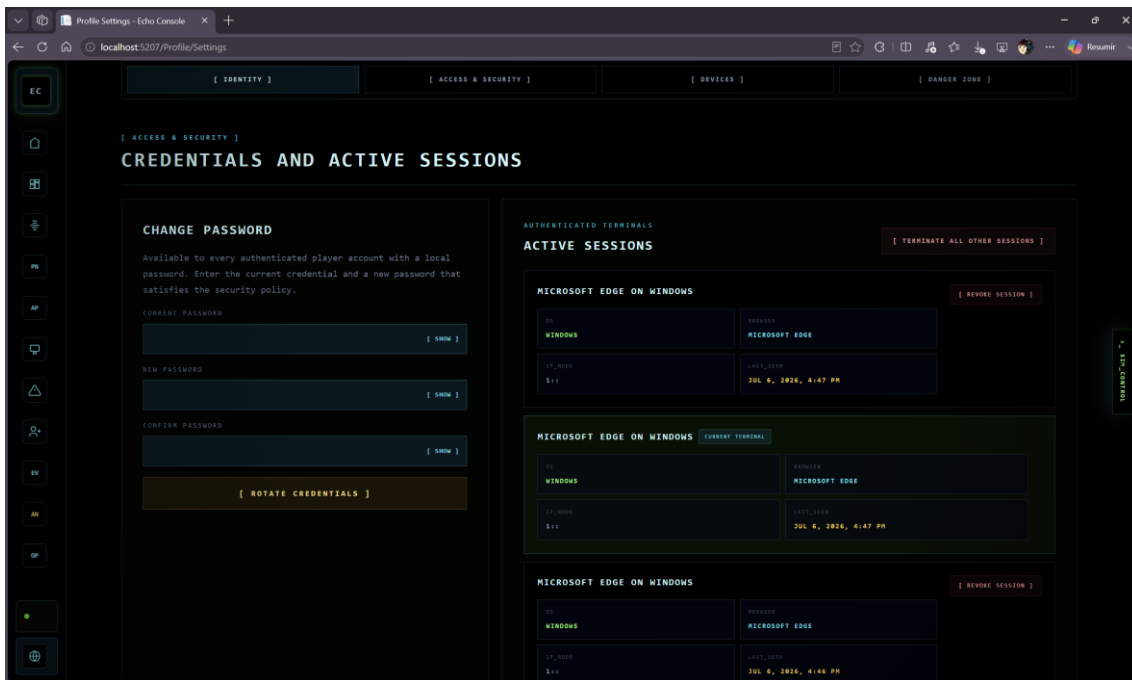
STARTED AT	DURATION	SCENE / PHASE	STATUS	BUILD	DEVICE	LAST SIGNAL	ACTIONS
2026-07-06 15:56	50m	Level_01 LevelProgression	LIVE	v2.1.0-Produccion	PC-Player-0004 c7b7d9d-1f57-470a-a808-e133180076c4	Just now	OPEN
2026-07-06 15:54	52m	Combat_Zone EnemyEncounter	LIVE	v1.1.2-Beta-Testing	PC-Player-0003 2d7430d8-5f50-49f9-a4ab-v980-831f60c	Just now	OPEN
2026-07-06 15:46	1h	Kitchen_Hideout Investigation	LIVE	v2.1.0-Produccion	PC-Player-0001 0946a4d2-cf97-4c30-b053-1225200479a3	Just now	OPEN
2026-07-04 09:08	43m	Combat_Zone EnemyEncounter	COMPLETED	v1.1.2-Beta-Testing	PC-Player-0024 95d6d8d-1008-49a8-b87a-8ea71340112	2d ago	OPEN
2026-07-04 09:07	43m	Diner_Main ServiceFloor	COMPLETED	v1.1.2-Beta-Testing	PC-Player-0048 5a8800fa-c0a7-4c0a-8a0a-4aff3177220	2d ago	OPEN
2026-07-04 09:05	45m	Level_01 LevelProgression	COMPLETED	v2.1.0-Produccion	PC-Player-0004 c7b7d9d-1f57-470a-a808-e133180076c4	2d ago	OPEN
2026-07-04 09:04	47m	Level_01 LevelProgression	COMPLETED	v2.1.0-Produccion	PC-Player-0019 a7e7a3c3-3a07-4a17-40a2006720a8	2d ago	OPEN
2026-07-04 09:03	47m	Kitchen_Hideout Investigation	COMPLETED	v2.0.0-RetroUI	PC-Player-0035 80e11708-3a60-40a8-a108-7a70257c82208	2d ago	OPEN
2026-07-04 09:03	48m	Level_01 LevelProgression	COMPLETED	v2.1.0-Produccion	PC-Player-0013 0c40c444-c25a-4311-ba08-4180c5a633a	2d ago	OPEN
2026-07-04 09:02	48m	Combat_Zone EnemyEncounter	COMPLETED	v1.1.2-Beta-Testing	PC-Player-0003 2d7430d8-5f50-49f9-a4ab-v980-831f60c	2d ago	OPEN

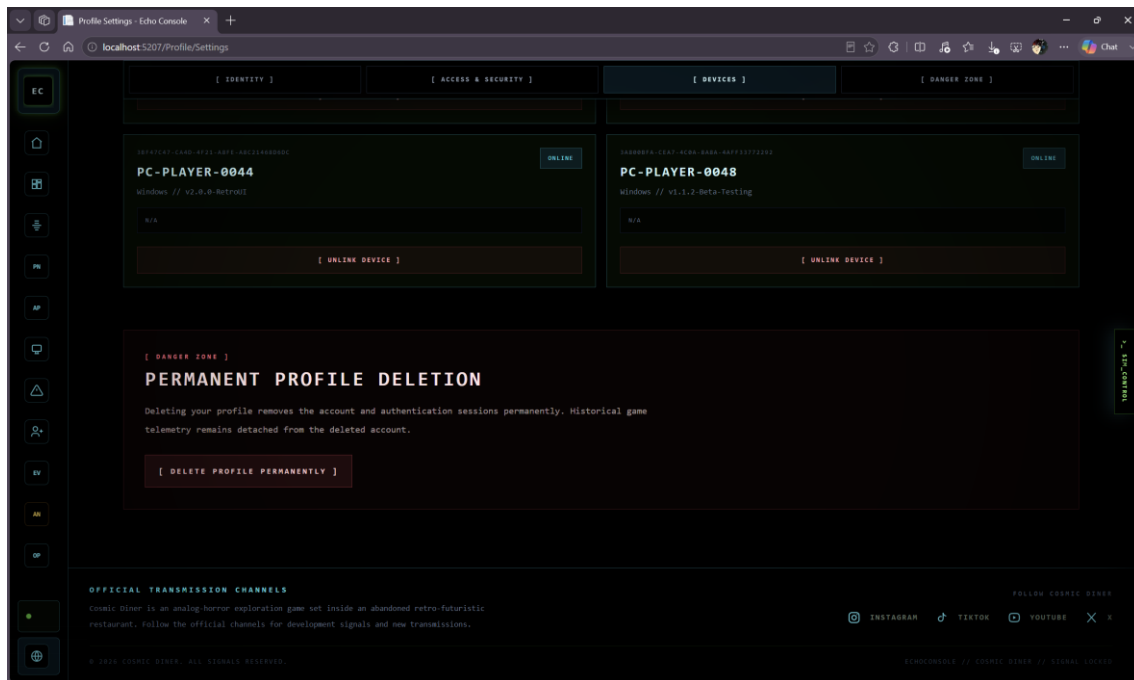
Profile/Sessions.cshtml: Historial de sesiones del usuario autenticado.



Profile/Settings.cshtml: Configuración del usuario con identidad, seguridad, flota y apariencia.

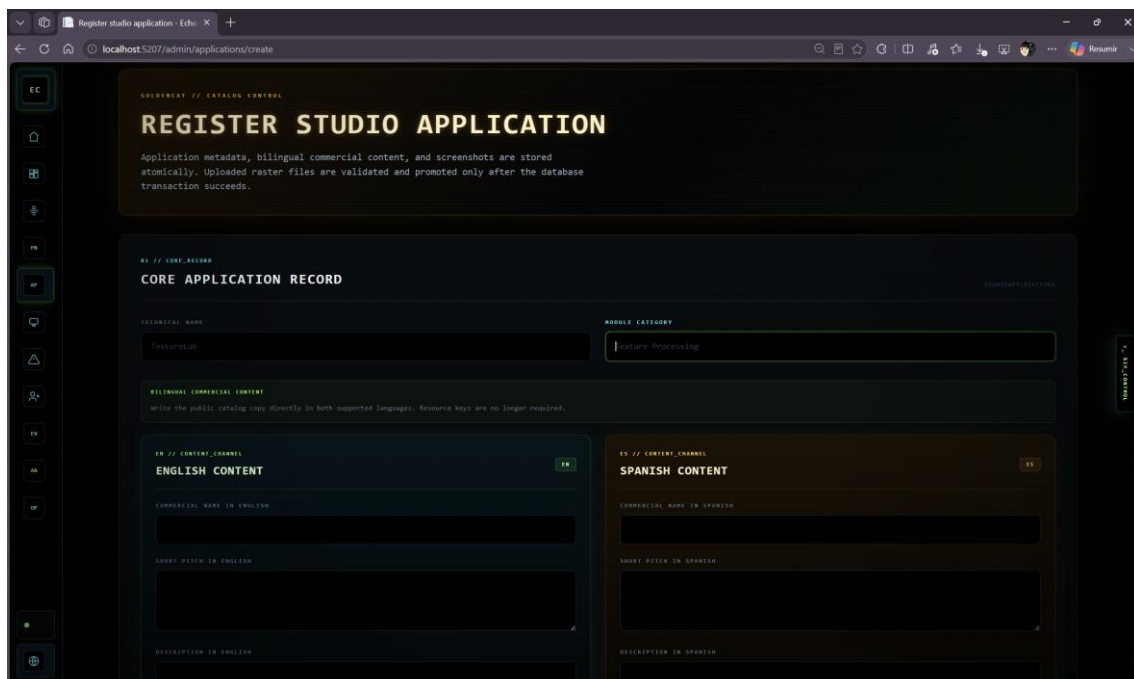


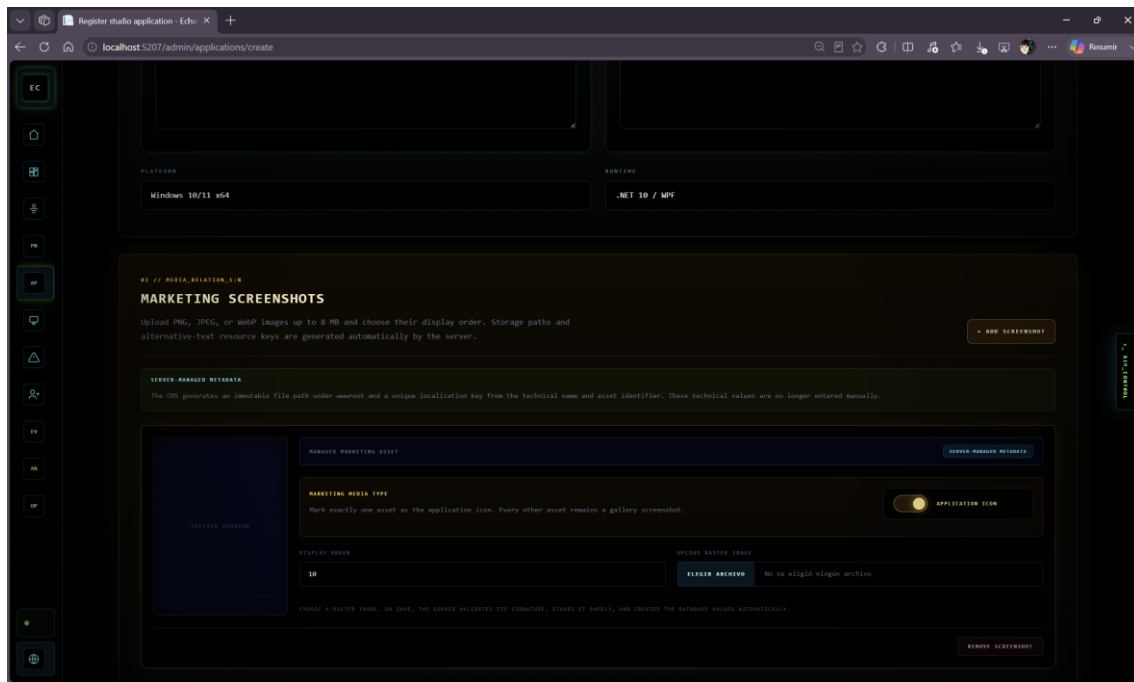




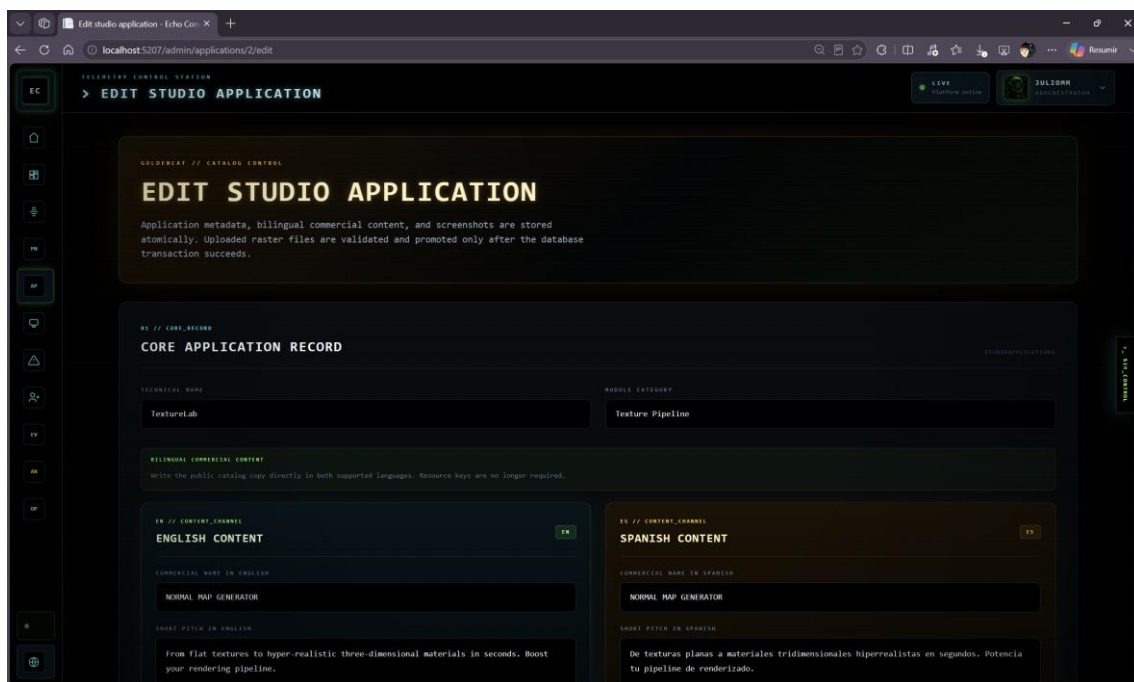
Módulo StudioApplications

StudioApplications/Create.cshhtml: Formulario de creación del CMS GoldenCat con soporte bilingüe y media.

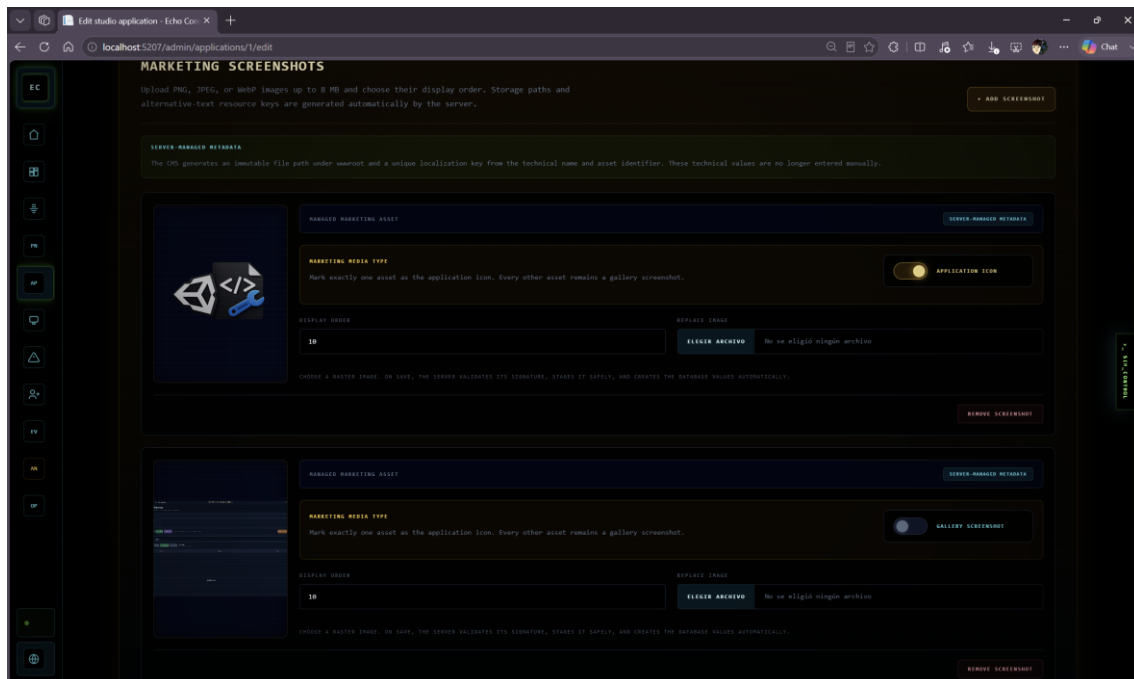
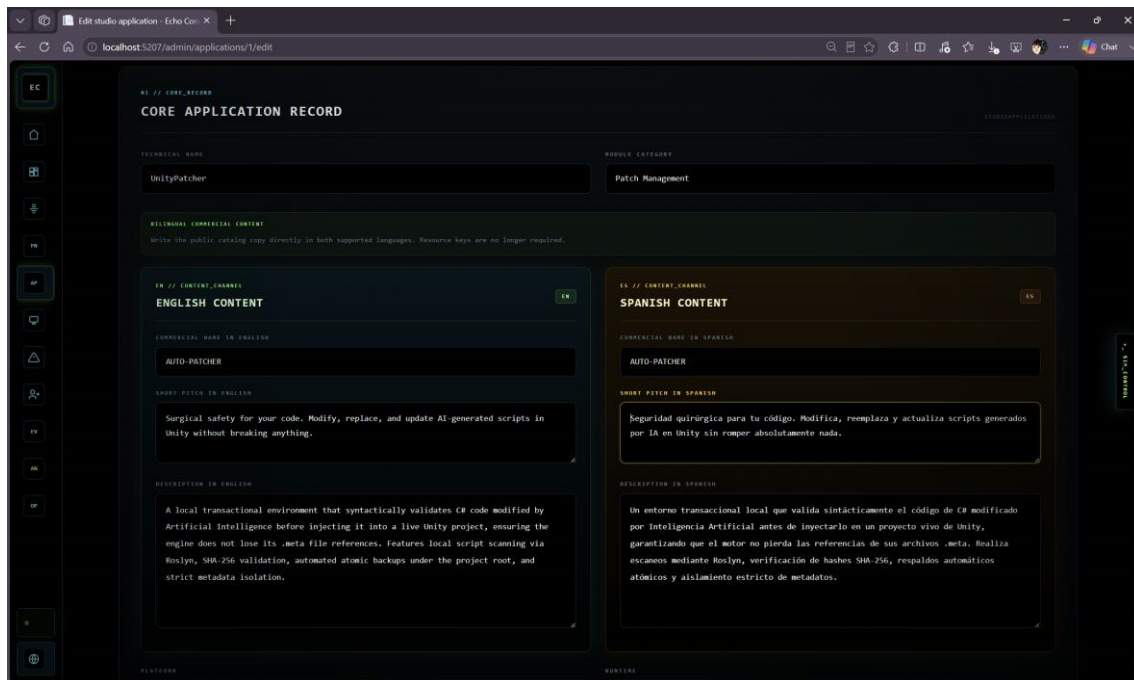




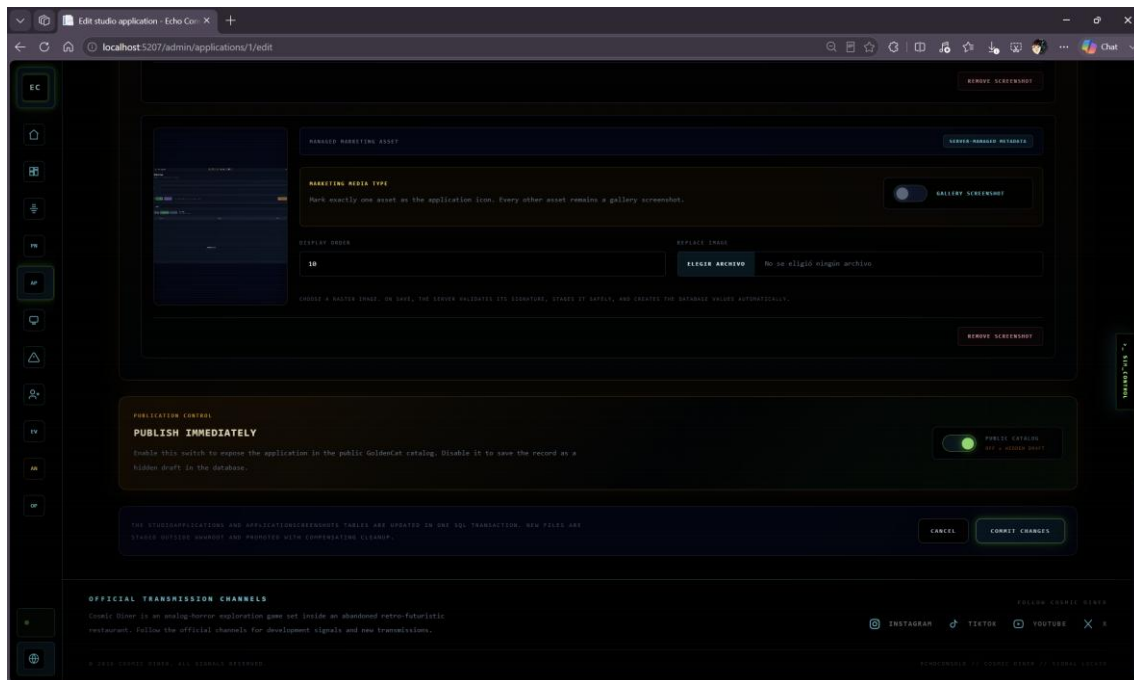
StudioApplications/Edit.cshtml: Edición de aplicaciones del catálogo GoldenCat con soporte completo de CRUD.



StudioApplications/Index.cshtml: Listado administrativo del catálogo GoldenCat con gestión de estado y publicación.

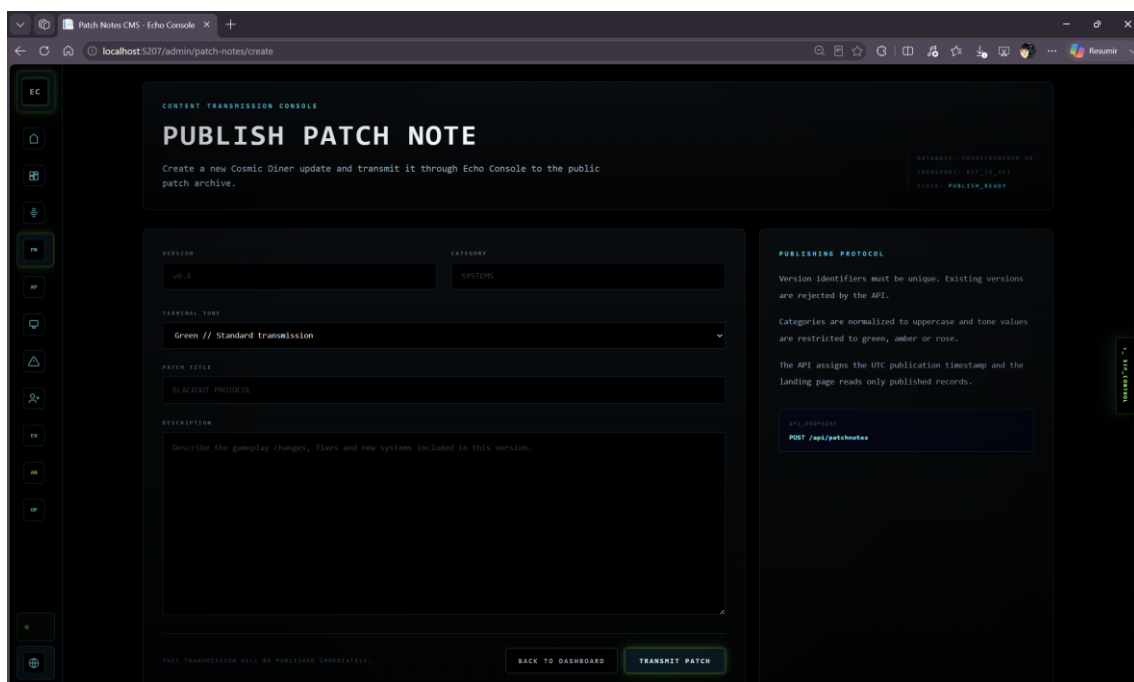


StudioApplications/_ApplicationForm.cshtml: Formulario principal bilingüe con editor de media y configuración de iconos y screenshots.

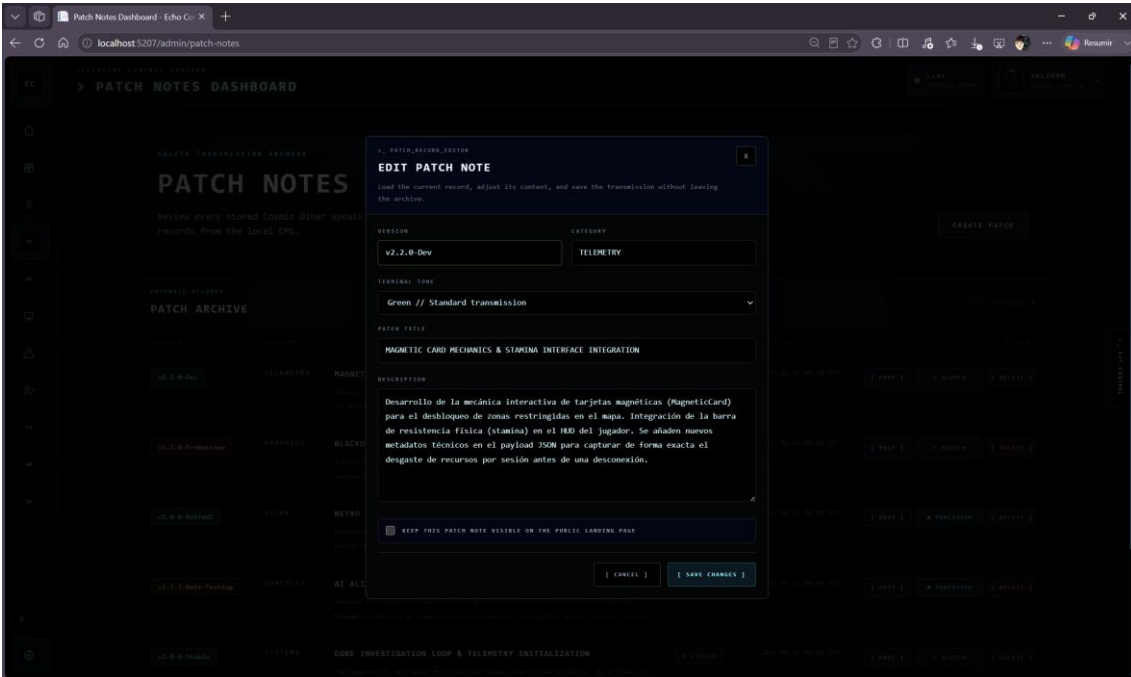
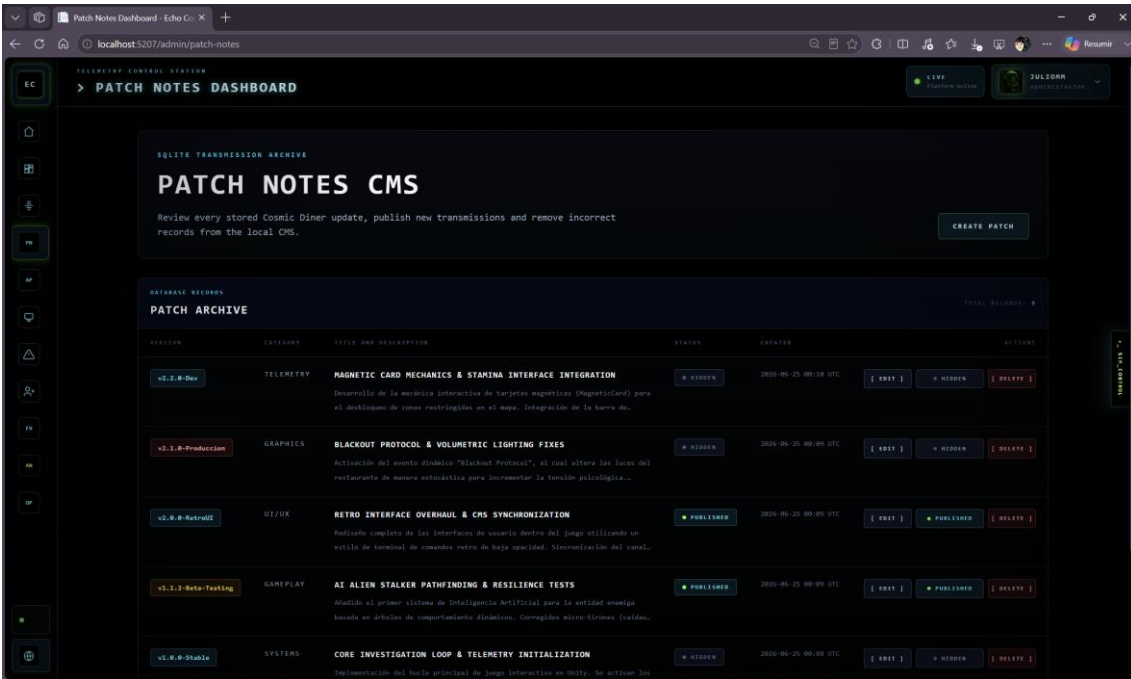


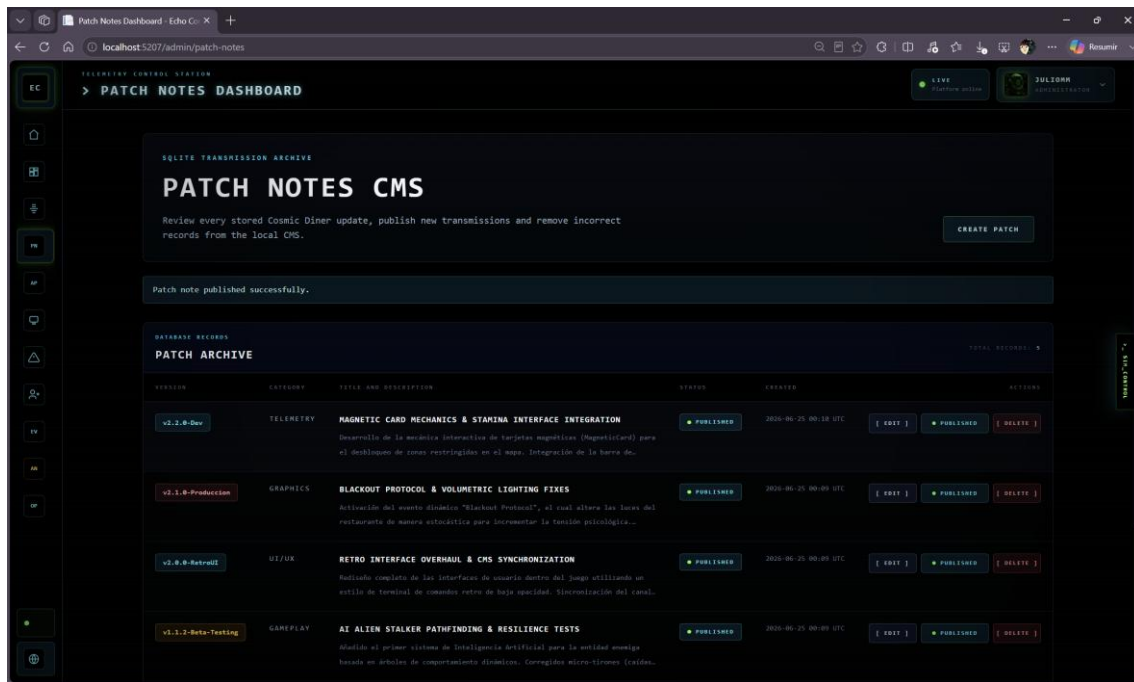
Módulo PatchNotesAdmin

PatchNotesAdmin/Create.cshtml: Formulario de creación de notas de parche.



PatchNotesAdmin/Index.cshtml: Catálogo administrativo de notas de parche con CRUD y publicación.

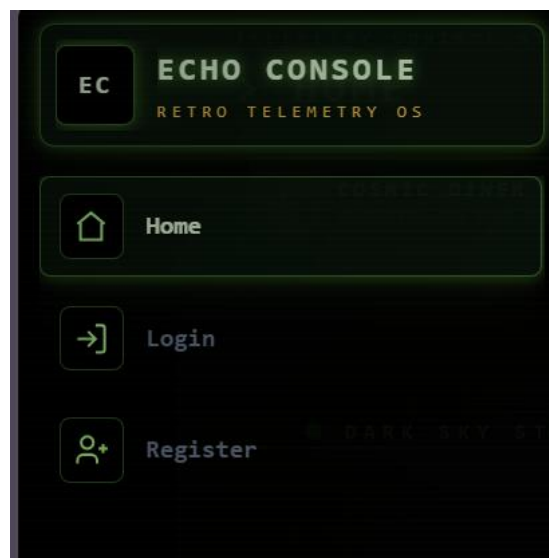




Módulo Shared

Shared/Error.cshtml: Vista de error global del sistema.

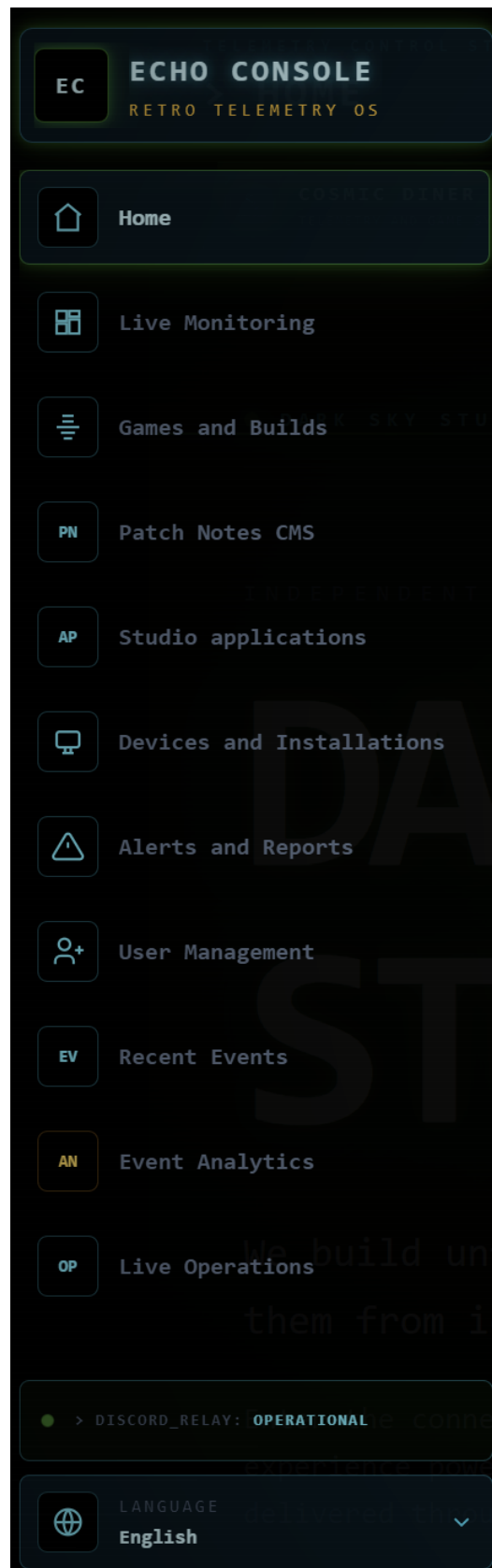
Shared/_AuthLayout.cshtml: Layout de autenticación con estructura base para login y registro.



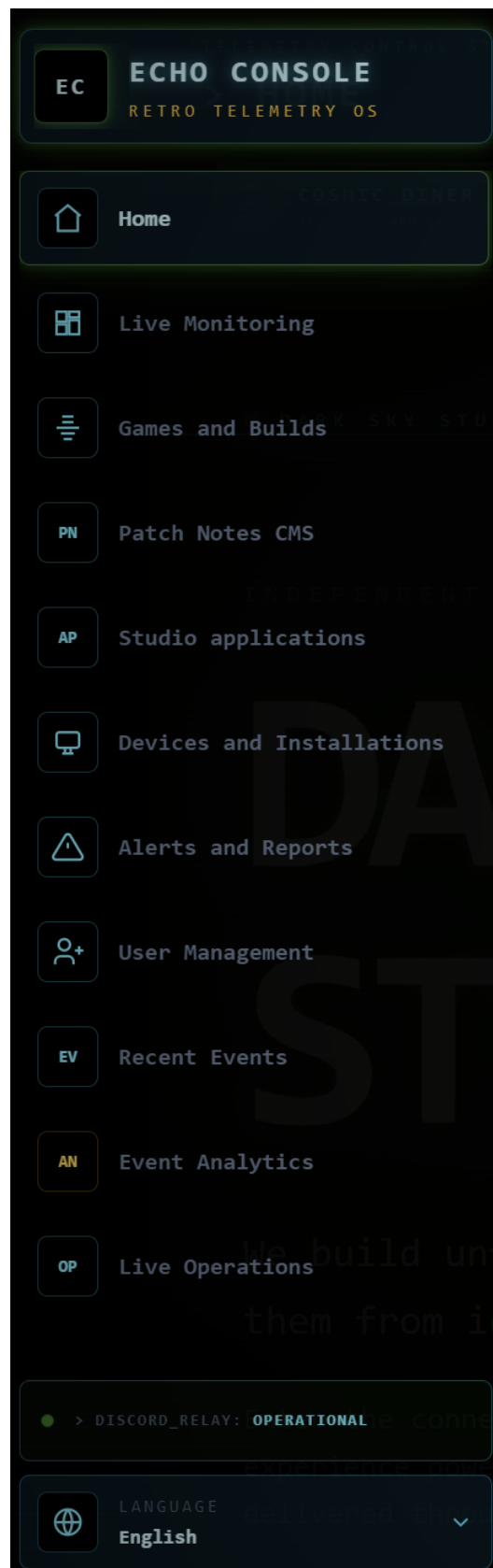
Shared/_Footer.cshtml: Pie de página global del sistema.



Shared/_Layout.cshtml: Layout principal del shell visual de la aplicación.



Shared/_Sidebar.cshtml: Sidebar dinámico basado en claims y contexto del usuario.



Shared/_SimulationOrchestrator.cshtml: Panel de orquestación de simulaciones administrativas.

>_ TELEMETRY_SIMULATION_OS

CLOSE [X]

SIMULATION ORCHESTRATOR

CONTROL CHANNEL // ADMIN ONLY

[STEP_01: LOAD_CONFIGURATION]

MODULE_04

ACTIVE SESSION CONTROL

[STABLE]

ACTIVE_REAL

0

ACTIVE_SIMULATED

5

CONNECTED USER TARGET

5

APPLY

CHURN_INTENSITY // VOLATILITY

MEDIUM // MEDIA

RAMP_SPEED // OSCILLATION // CHURN_RATE // HEARTBEAT_FAILURES

ORGANIC STOCHASTIC MODE

Random entry, scene transition and exit pulses every 4-8 seconds.

PERSISTENT MODE REMAINS ACTIVE ACROSS PAGE NAVIGATION UNTIL TERMINATED.

[TERMINATE_SIMULATION]

[STEP_02: COMPONENT_MATRIX]

COMPONENT_MATRIX

✓

SESSIONS

✓

INSTALLATIONS

✓

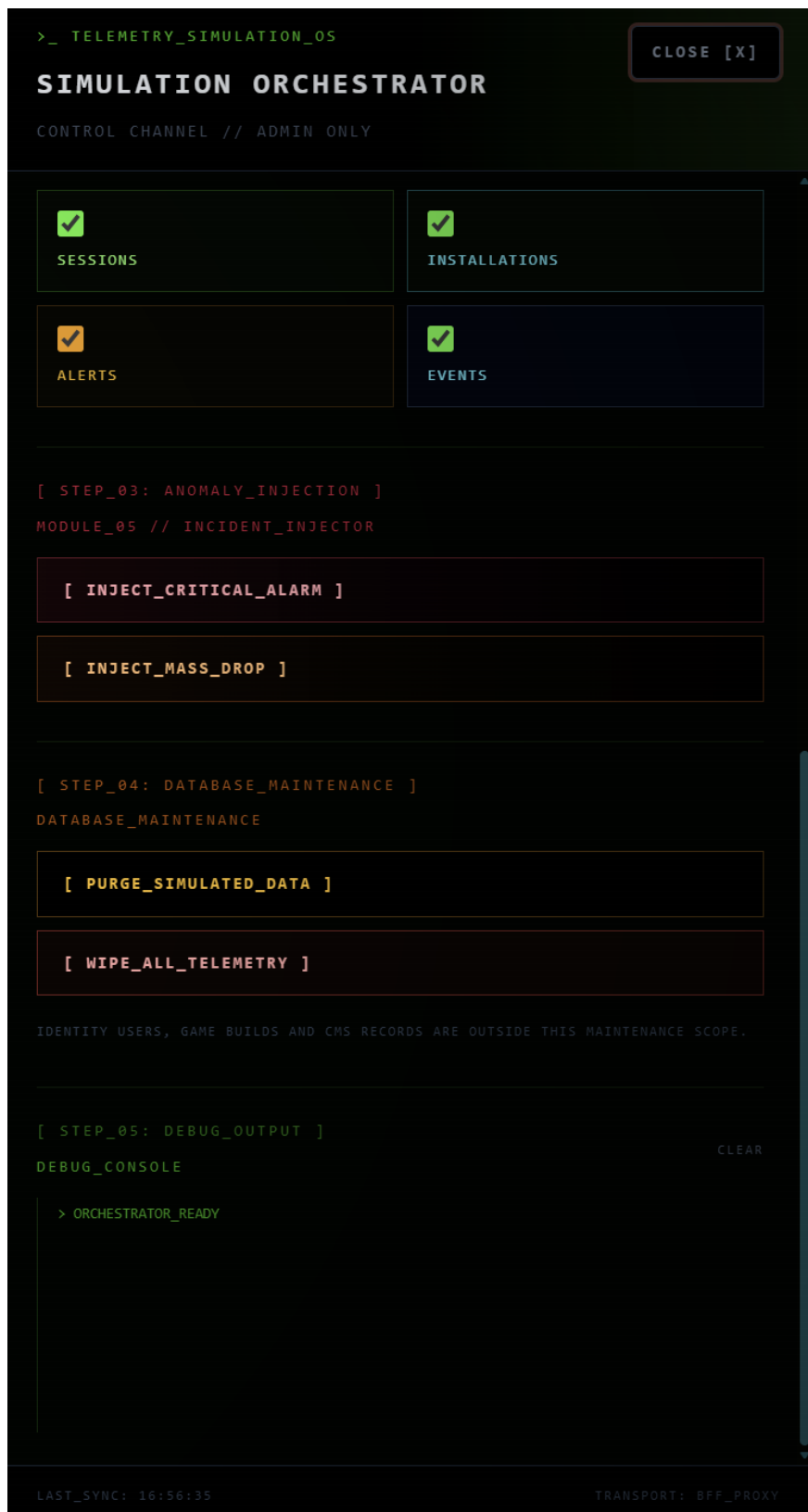
ALERTS

✓

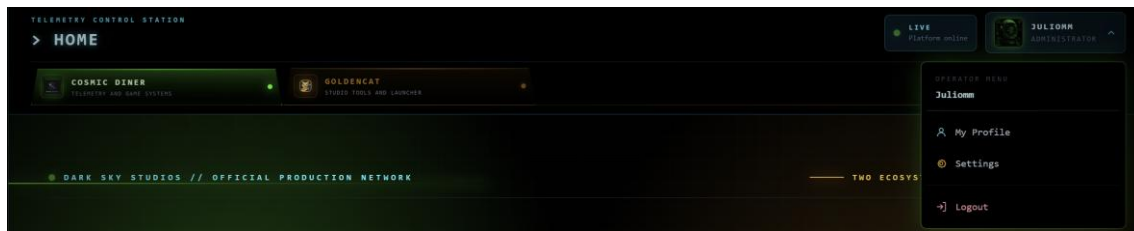
EVENTS

LAST_SYNC: 16:56:22

TRANSPORT: BFF_PROXY



Shared/_Topbar.cshhtml: Barra superior con estado del sistema, navegación y control de usuario.



Shared/_ValidationScriptsPartial.cshtml: Scripts de validación cliente para formularios Razor.

Operación, telemetría, alertas y administración

DashboardController es Admin-only. Index obtiene el modelo inicial; Overview y LiveSessions exponen fragmentos/datos para refresco. El dashboard presenta KPIs de instalaciones, sesiones y alertas, mientras AdminTelemetryHub empuja cambios. Los parciales *_KpiCardsRow* y *_LiveSessionsTable* aíslan regiones de actualización.

LiveOperationsController y *LiveOperationsService* consolidan estado de instalaciones y sesiones en un snapshot. La ruta */Admin/LiveOperations/Snapshot* permite polling o actualización del tablero. La lógica operacional distingue estado reciente, sesión activa, build y propiedad del dispositivo.

SessionEvents ofrece búsqueda paginada, detalle de timeline y purga. *SessionEventAnalytics* agrega buckets por EventType y puntos temporales. La separación entre consulta operacional y analítica evita que la tabla de eventos de alto volumen se renderice completa en una sola vista.

Alerts combina ingestión pública y administración. Los clientes crean SystemAlert; la API normaliza ErrorCode, resuelve severidad y build, persiste y emite SignalR. El panel Admin lista, filtra y resuelve. AlertTypeDefinitions es un catálogo editable; al eliminar un tipo, las alertas se reasignan a UNCLASSIFIED dentro de una transacción ejecutada por la estrategia resiliente.

BuildsAdminController administra GameBuild. VersionNumber es único. Al activar un build desactiva el anterior dentro de IsolationLevel.Serializable. Al renombrar una versión actualiza referencias en Installations y GameSessions mediante ExecuteUpdateAsync. La eliminación rechaza el build activo o referenciado por telemetría.

Installations almacena hardware, plataforma, build, estado y ownership. El endpoint cliente realiza upsert por InstallationId. El panel Admin añade alias/estado operacional. La

relación con User usa SetNull y con GameSession usa Restrict, por lo que la evidencia histórica se preserva.

UsersController es Admin-only y consume /api/admin/users. La API devuelve páginas de cuentas, roles, estados y hardware vinculado. El código de dominio permite Admin, Moderator y Viewer y estados Active/Suspended. El acceso operativo sigue concentrado en Admin.

El módulo /admin/simulation es una herramienta de demostración y QA. Permite consultar estado, reconciliar módulos, emitir pulsos, inyectar alertas críticas, simular caídas masivas y purgar datos simulados o telemetría. Las operaciones destructivas usan antiforgery, confirmaciones y la API key administrativa.

PatchNotesAdminController expone CRUD sobre el CMS SQLite de la API. La vista pública Home consume solo notas publicadas. Version es única; Title/Description tienen longitudes y la fecha se normaliza a UTC.

Integraciones con API's

Discord API / Webhooks

La integración se implementa como webhook HTTPS, no como bot OAuth completo. DiscordOptions se valida al iniciar: PollIntervalSeconds entre 1 y 60 y BatchSize entre 1 y 50. El worker envía embeds con origen, build, fecha UTC, tipo, severidad y mensaje. allowed_mentions.parse es vacío para impedir menciones masivas.

GitHub API

La integración confirmada es la descarga del instalador desde GitHub Releases privado. Se usa un PAT de lectura configurado en GitHubRelease:Token, un User-Agent propio y la API REST. El portal no expone la URL privada ni el PAT. El código valida Owner/Repository con regex, AssetName como .exe, tamaño máximo entre 1 byte y 1 GiB y timeout entre 5 y 120 s.

No se encontró una integración de sincronización de código fuente ni una API de catálogo consumida por el launcher. El catálogo visible se renderiza en el servidor MVC. Cualquier consumo remoto por GoldenCat debe considerarse una extensión futura salvo evidencia adicional.

Google APIs

La única infraestructura Google confirmada es Google OAuth 2.0 para autenticación. Se habilita condicionalmente cuando Authentication:Google:ClientId y ClientSecret existen, usa la cookie externa, guarda tokens y mapea verified_email. No hay evidencia de Drive, Analytics, Cloud Storage ni otras APIs Google.

SMTP

SmtpEmailSender requiere Host, Port, Username y Password, permite FromEmail/FromName y SSL. Envía OTP de seis dígitos, enlaces de reset y notificaciones de cambio de contraseña. Los mensajes codifican alias y contenido HTML; los logs incluyen UserId, email y expiración, por lo que la retención debe tratarse como dato personal.

SignalR

EchoConsole.Api publica eventos en TelemetryHub. TelemetryRelayService conecta como cliente con la API key y los retransmite a AdminTelemetryHub y ProfileTelemetryHub. AdminTelemetryHub exige rol Admin; ProfileTelemetryHub exige autenticación y usa grupos por UserId. Esta topología evita que el navegador conozca la API key.

Atributos de calidad, observabilidad, rendimiento, mantenibilidad y recuperabilidad

Atributo	Mecanismos
Seguridad	Identity, cookies endurecidas, API key, tokens hashados, rate limit, antiforgery, validación JSON
Disponibilidad	Retry SQL principal, outbox Discord, workers, SignalR reconnect, separación de hosts
Rendimiento	DbContextPool API, AsNoTracking, índices, paginación, streaming y compresión
Escalabilidad	Hosts separables, consultas paginadas, rate limit
Mantenibilidad	Módulos claros y servicios especializados
Testabilidad	DI, interfaces y DTOs

Observabilidad	ILogger, SignalR, KPIs, health implícito, outbox status
Localización	RESX UI + SQL copy bilingüe
Recuperabilidad	Transacciones, compensación de archivos

Repositorio

Enlace: <https://github.com/Juliomm8/EchoConsole>

Referencias bibliográficas

- Aguas Bucheli, L. F. (2024b). *Estructuras de Datos para Todos: Ejercicios y Teoría con Java*. Our Knowledge Publishing.
- Aguas Bucheli, L. F. (2023a). *Dominando la Programación Orientada a Objetos con Java en NetBeans*. Our Knowledge Publishing.
- Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.
- Fortune Business Insights. (2026, 6 abril). Tamaño, participación y crecimiento del mercado de juegos en línea | Pronóstico [2034]. <https://www.fortunebusinessinsights.com/es/online-gaming-market-112179>
- Horstmann, C. S. (2018). *Core Java Volume I--Fundamentals* (11th ed.). Prentice Hall.
- Kaufmann, D. (2022, 17 junio). Perfiles de jugadores: Entendiendo la psicología de los videojuegos. Dr. Gameology. <https://drgameology.com/gamer-personas-understanding-game-psychology/>
- Pereira, P. (2023, 13 julio). La Importancia de los Datos Precisos en el Comercio Electrónico: Impulsando Decisiones Estratégicas - Lett Digital. Lett Digital. <https://lett.digital/es/product-updates/la-importancia-de-los-datos-precisos-en-el-e-commerce-impulsando-decisiones-estrategicas-experiencia-del-cliente-y-reputacion-de-la-marca/>
- Rosenfelder, S. (2024, 1 enero). Análisis de datos en videojuegos: por qué importan las cifras. AppsFlyer. <https://www.appsflyer.com/blog/measurement-analytics/gaming-analytics/>
- Safdie, S. (2025, 31 octubre). The Environmental Impact of Video Games. Greenly. <https://greenly.earth/en-us/leaf-media/data-stories/the-environmental-impact-of-video-games>
- Sierra, K., & Bates, B. (2014). *Head First Java*. O'Reilly Media.